

Vigilant OneTimer

Adaptive Inference Governor for Edge AI - Produkt-, Lasten-, Pflichten- und Entwicklungsheft

Vigilant e.K. - Stuttgart, Deutschland

31. August 2026

Inhaltsverzeichnis

Dokumentstatus	6
Executive Summary	7
Reach-out Executive Summary - Deutsch	7
Reach-out Executive Summary - English	7
Ein Satz fuer Entwickler	7
Ein Satz fuer Management	7
Was OneTimer am Ende fuer einen Roboter bewirkt	7
1. Problemdefinition in einfachen Worten	8
1.1 Warum normale Warteschlangen bei Robotern problematisch sind	8
1.2 Die Garbage-Collector-Analogie	8
1.3 Warum das Problem mit VLM/LLM plus Detektor größer wird	9
2. Produktthese und Marktthese	9
2.1 Was bereits existiert	9
2.2 Warum die technische Nachfrage plausibel ist	9
2.3 Deutschland als Design- und Startmarkt	10
3. Konkurrenzanalyse und reale Abgrenzung	10
3.1 NVIDIA Triton	10
OneTimer vs. Triton	10
3.2 NVIDIA Holoscan	11
3.3 REEF und aktuelle Forschung	11
3.4 Wettbewerbsmatrix	11
3.5 Was OneTimer nicht behaupten darf	12
4. Kundennutzen mit Zahlen - Rechenbeispiele und Zielmetriken	12
4.1 Beispiel: Frische statt FIFO-Rückstand	12
4.2 Beispiel: vermiedene stale GPU-Arbeit	12
4.3 Beispiel: geschütztes Compute-Budget	13
4.4 Produkt-Go/No-Go-Zielwerte	13
4.5 Wirtschaftlicher Effekt - Szenarien	13
5. Produktarchitektur - endgültige MVP-Entscheidung	14
5.1 OneTimer wird Triton zunächst nicht neu schreiben	14
5.2 Warum der MVP-Core in Rust gebaut werden sollte	15
5.3 Data Path und Control Path trennen	15

6. Plug-and-Play-Integration	15
6.1 Zielintegration	15
6.2 Beispielkonfiguration	16
6.3 Minimaler Nutzerworkflow	17
6.4 Docker-Quickstart	17
7. Funktionale Produktanforderungen - Lastenheft	17
7.1 Muss-Anforderungen MVP	17
7.2 Soll-Anforderungen nach MVP	18
8. Nichtfunktionale Anforderungen	18
8.1 Performance	18
8.2 Stabilität	18
8.3 Sicherheit	18
8.4 Wartbarkeit	19
9. Pflichtenheft - interne Systemarchitektur	19
9.1 Komponenten	19
9.2 Repository-Struktur	20
9.3 Kern-Datentypen	21
9.4 Single-owner Scheduler Actor	21
10. Tiefe Scheduling-Theorie	22
10.1 Zielgröße: Useful Timely Inference	22
10.2 Latency ist nicht Age of Information	22
10.3 Drei Stufen der Stale-Pruefung	22
Stufe A - Ingress Supersession	22
Stufe B - Pre-dispatch Freshness	23
Stufe C - Completion Validity	23
10.4 Slack / Laxity	23
10.5 Earliest Deadline First als Basis, nicht als Endloesung	23
10.6 Lexikographische Zielreihenfolge	23
10.7 Warum absichtliches Idle korrekt sein kann	24
10.8 Periodische Future Arrivals	24
10.9 Demand-/Utilization-Analyse offline	24
10.10 Non-preemptive Blocking	25
10.11 Protected Slack Server	25
11. Stale Work Collector im Detail	25
11.1 LATEST	25
11.2 LATEST_PER_KEY	25
11.3 FIFO	26
11.4 NEVER_DROP	26
11.5 SAMPLE / DECIMATE - spaeter	26
12. Model Variant Selection	26
12.1 Logisches vs. physisches Modell	26
12.2 Qualitätswert im MVP	27
12.3 Auswahlregel	27
12.4 Hysterese gegen Modell-Flattern	27
12.5 Stateful Models	27
13. Runtime Profiling und Interferenz	27
13.1 Warum Durchschnittslatenz nicht reicht	27
13.2 Online Runtime Estimator	28
13.3 Langsame Margin-Anpassung	28
13.4 Interference Matrix	28

13.5 Profile Fingerprint	28
14. Overload Control und Graceful Degradation	29
14.1 Zustandsmaschine	29
14.2 Beispielaktionen	29
NORMAL	29
FRESHNESS_PRESSURE	29
DEGRADED	29
REJECT_BEST_EFFORT	29
PROTECTED_ONLY	29
14.3 Hysterese	29
15. VLM/LLM-Sonderfall	30
15.1 Problem	30
15.2 MVP-Verhalten	30
15.3 Vollausbau: kooperative Quanten	30
16. Protokoll und API	30
16.1 Open Inference Protocol v2	30
16.2 OneTimer Parameter	30
16.3 Compatibility Mode	31
17. Shared Memory und Copy-Vermeidung	31
17.1 Warum dies früh relevant wird	31
17.2 Triton Shared Memory	31
17.3 MVP-Stufen	31
M0	31
M1	31
M2 x86/dGPU	31
M3 Jetson	31
18. Observability und Kennzahlen	31
18.1 Useful Inference Ratio	32
18.2 Stale Compute Waste	32
18.3 Age of Information	32
19. Benchmark- und Falsifikationsplan	32
19.1 Baselines	32
19.2 Benchmark-Framework	32
19.3 Erste Modellgruppe	32
19.4 Lastprofile	33
19.5 Kernvergleich A - Freshness	33
19.6 Kernvergleich B - Protected versus Best Effort	33
19.7 Kernvergleich C - Varianten	33
19.8 Kill-Kriterien	34
20. Open-Source- und Lizenzstrategie	34
20.1 OneTimer Core	34
20.2 Enterprise/Open-Core	34
20.3 Triton	34
20.4 Open Inference Protocol	35
20.5 DISB	35
20.6 REEF	35
20.7 NVIDIA TensorRT / NGC	35
20.8 Lizenz-Allowlist	35
21. Lizenz-, Security- und Supply-Chain-Tests	36

21.1 Rust Dependencies	36
21.2 SPDX/REUSE	36
21.3 SBOM	36
21.4 Vulnerability Scan	36
21.5 FOSS Review vor öffentlichem Release	36
22. Security Threat Model MVP	37
22.1 Angriffsflächen	37
22.2 Schutzregeln	37
22.3 Fuzzing	37
23. Konfigurationsvalidierung und onetimer doctor	37
24. Arbeitspakete bis Vollausbau	38
WP0 - Repository, Build, CI, Lizenzgrundlage	38
WP1 - Discrete Event Scheduler Simulator	38
WP2 - Queue Policies und Stale Work Collector	38
WP3 - Deadline, Slack, Feasibility	39
WP4 - Schedulability/Doctor Analyzer	39
WP5 - Overload State Machine	39
WP6 - Model Variant Resolver	39
WP7 - OIP gRPC Gateway - transparente Stufe	39
WP8 - Triton Backend Adapter	39
WP9 - Echtzeit-Policies im Gateway	40
WP10 - Single-Model Profiler	40
WP11 - Online Runtime Estimator	40
WP12 - Interference Profiler	40
WP13 - Metrics / Prometheus	40
WP14 - Shared-Memory Compatibility	40
WP15 - Benchmark Harness / DISB Adapter	40
WP16 - Tuned Triton Baseline	40
WP17 - Freshness Benchmark	41
WP18 - Protected-vs-Best-Effort Benchmark	41
WP19 - Variant Benchmark	41
WP20 - Stress, Property, Fuzz	41
WP21 - Docker/Quickstart/Product Preview	41
WP22 - Jetson Port	41
WP23 - HTTP/OIP	41
WP24 - Stateful/Sequence Support	41
WP25 - DAG / End-to-End Deadlines	42
WP26 - Cooperative VLM/LLM Adapter	42
WP27 - Optional Native Executor	42
WP28 - Experimental Preemption Backend	42
WP29 - Multi-Accelerator	42
WP30 - Fleet/Enterprise	42
25. Milestones und harte Go-Gates	42
Milestone M0 - Theorie	42
Milestone M1 - Transparent Proxy	42
Milestone M2 - Profiling	42
Milestone M3 - Falsifikation	43
Milestone M4 - Product Preview	43
Milestone M5 - Erweiterungen	43
26. Coding-Agent-Protokoll	43
26.1 Grundregel	43
26.2 PR-/Commit-Reihenfolge	43

26.3 Definition of Done fuer jeden Coding-Agent-Task	43
26.4 Coding-Regeln	44
27. Master Prompt fuer einen LLM-Coder	44
28. Testkatalog	45
28.1 Golden Test G-001 - Latest supersedes queued	45
28.2 G-002 - Running is not magically cancelled	45
28.3 G-003 - FIFO order	45
28.4 G-004 - NeverDrop overflow	45
28.5 G-005 - Deadline from generation time	45
28.6 G-006 - Variant	45
28.7 G-007 - Known future protected arrival	46
28.8 G-008 - Overload bounded	46
28.9 G-009 - Runtime degradation	46
28.10 G-010 - Profile stale	46
28.11 G-011 - Stateful cannot Latest	46
28.12 G-012 - Time overflow	46
29. Performanceoptimierungsplan	46
29.1 Nicht rechnen	46
29.2 Nicht zu frueh rechnen	46
29.3 Richtige Variante	46
29.4 Konkurrenz kennen	46
29.5 Kopien vermeiden	46
29.6 Hot Path vereinfachen	47
29.7 Erst danach Mikrooptimieren	47
30. Stabilitaetsoptimierungsplan	47
30.1 Deterministischer Core	47
30.2 Replayability	47
30.3 Profile Circuit Breaker	47
30.4 Backend Circuit Breaker	47
30.5 Config Atomicity	47
31. Was Open Source uebernommen werden sollte - und was nicht	48
31.1 Direkt sinnvoll	48
Open Inference Protocol	48
Triton als Execution Backend	48
DISB	48
31.2 Nur als Forschung/Referenz	48
REEF	48
EdgeServing	48
Holoscan	48
31.3 Nicht sinnvoll im MVP	48
32. Produktverpackung und Developer Experience	48
32.1 README Above the Fold	48
32.2 Demo	49
33. Go-to-Market-Story	49
33.1 Was nicht verkauft wird	49
33.2 Was verkauft wird	49
34. Design-Partner-Fragen	49
35. Risiken	50

R1 - Problem existiert, aber interne Eigenentwicklung ist "gut genug"	50
R2 - Triton/Holoscan schliessen Featureluecke	50
R3 - Proxy-Overhead vernichtet Vorteil	50
R4 - Varianten sind fachlich nicht austauschbar	50
R5 - GPU-Preemption zu begrenzt	50
R6 - Realer Workload besitzt kaum stale Arbeit	50
R7 - Safety/Automotive Erwartungen	50
R8 - Lizenz-/Redistributionsfehler	50
36. Vollausbauvision	50
37. Endgueltige Produktentscheidung	51
38. Die eine entscheidende Hypothese	52
Anhang A - Quellen und Recherchebasis	53
Anhang B - Begriffsdefinitionen	54
Anhang C - Release-Checkliste v0.1	54
Anhang D - OneTimer v0.1 Erfolgskriterien auf einer Seite	55

Dokumentstatus

Hersteller: Vigilant e.K., Stuttgart, Deutschland

Produkt-/Arbeitsname: Vigilant OneTimer

Technischer Untertitel: Adaptive Inference Governor for Edge AI

Dokumentversion: 1.0

Status: Produktdefinition und Implementierungsbasis

Ziel dieses Dokuments: OneTimer so vollständig spezifizieren, dass ein erfahrenes Engineering-Team oder ein Coding-Agent das System schrittweise implementieren, benchmarken, falsifizieren und bis zu einem produktionsfähigen Open-Core-Produkt ausbauen kann.

Wichtiger Statushinweis: Alle im Dokument genannten Leistungswerte, soweit nicht ausdrücklich als externe Messwerte mit Quelle gekennzeichnet, sind **Zielwerte, Rechenbeispiele oder Validierungsschwellen**. Sie sind keine bereits gemessenen Leistungsbehauptungen von OneTimer. Vor öffentlicher Verwendung müssen die betreffenden Aussagen durch reproduzierbare Benchmarks ersetzt oder als Ziel/Hypothese gekennzeichnet werden.

Rechtlicher Hinweis: Die Lizenz- und Compliance-Empfehlungen in diesem Dokument sind technische Produktplanung und keine Rechtsberatung. Vor kommerzieller Distribution, OEM-Verträgen, Markennutzung und Redistribution fremder Software ist eine formale juristische Prüfung erforderlich.

Executive Summary

Reach-out Executive Summary - Deutsch

Vigilant OneTimer ist eine schlanke, kompatible Steuerungsschicht für KI-Inferenz auf Edge-Systemen, auf denen mehrere Modelle dieselbe GPU teilen. Moderne Roboter, autonome Maschinen und mobile KI-Systeme führen gleichzeitig Objekterkennung, Tiefenschätzung, Segmentierung, Pose-Erkennung und zunehmend VLM-/LLM-Workloads aus. Generische Inference Server sind darauf optimiert, Arbeit effizient abzuarbeiten; im physischen System ist jedoch nicht jede noch wartende Arbeit weiterhin wertvoll. Ein Kameraframe kann bereits veraltet sein, bevor seine Inferenz beginnt, und ein langsamer Best-Effort-Job kann eine zeitkritische Wahrnehmung unnötig verzögern. OneTimer sitzt als Drop-in-Governor vor einer vorhandenen NVIDIA-Triton-/TensorRT-Infrastruktur. Modelle und das standardisierte Inference-Protokoll bleiben bestehen. Der Entwickler definiert nur einfache Verträge wie "latest frame only", "30 ms Ziel", "protected" und optional mehrere Modellvarianten. OneTimer entfernt überholte Requests, bevor sie GPU-Zeit verbrauchen, lässt Arbeit nur zu, wenn sie noch sinnvoll abschließbar ist, schützt zeitkritische Inferenz vor weniger wichtiger Last und wählt bei Bedarf die bestmögliche noch rechtzeitig ausführbare Modellvariante. Das technische Ziel ist nicht lediglich mehr Durchsatz, sondern **mehr nützliche und rechtzeitige Inferenz pro vorhandener Edge-GPU**: frischere Wahrnehmungsdaten, weniger Deadline-Verletzungen, weniger verschwendete GPU-Zeit und kontrolliertes Verhalten unter Überlast. Der MVP wird bewusst gegen einen gut konfigurierten Triton-Stack validiert; erst bei einem großen, reproduzierbaren Vorteil wird der Produktumfang erweitert.

Reach-out Executive Summary - English

Vigilant OneTimer is a lightweight, protocol-compatible inference governor for edge systems where multiple AI models share the same GPU. Modern robots and autonomous machines may run object detection, depth estimation, segmentation, pose models and increasingly VLM/LLM workloads at the same time. Generic inference servers are designed to execute queued work efficiently, but in a physical system queued work can lose its value before it even starts. A camera frame may already be stale, while a long best-effort task can unnecessarily delay latency-sensitive perception. OneTimer sits in front of an existing NVIDIA Triton/TensorRT stack. Customers keep their models and the standard inference protocol; they define simple contracts such as "latest frame only", "30 ms target", "protected" and optional model variants. OneTimer removes obsolete requests before they consume GPU time, admits work only when it can still finish usefully, protects critical inference from lower-value load, and selects the highest-quality model variant that remains feasible. The goal is not simply higher throughput but **more useful, timely inference from the same edge GPU**: fresher perception, fewer deadline misses, less wasted compute and graceful, predictable behavior under overload. The MVP will be benchmarked against a strongly tuned Triton baseline, not default settings, and expanded only if the measured advantage is substantial and reproducible.

Ein Satz fuer Entwickler

Keep your models. Keep Triton. Tell OneTimer what must be fresh and what must be fast; OneTimer decides what should run now, what should wait, what should be replaced by newer work and which model variant is still feasible.

Ein Satz fuer Management

OneTimer soll aus derselben Edge-GPU mehr verlässlich nutzbare KI herausholen, indem es veraltete Berechnungen gar nicht erst ausführt und wichtige Wahrnehmungsmodelle bei Überlast schützt.

Was OneTimer am Ende fuer einen Roboter bewirkt

OneTimer macht einen Roboter nicht automatisch mechanisch schneller. Der primäre Effekt ist subtiler und für autonome Systeme oft wichtiger:

1. **Der Roboter arbeitet mit einem aktuelleren Weltzustand.** Alte Kameraframes werden nicht sinnlos nachgerechnet, wenn neuere Informationen bereits verfügbar sind.
2. **Zeitkritische Wahrnehmung wird stabiler.** Ein VLM oder eine andere langsame Hintergrund-Inferenz darf die regelmäßig benötigte Objekterkennung nicht unkontrolliert verdrängen.
3. **Überlast wird kontrolliert degradiert.** Statt dass alle Modelle gleichzeitig langsamer und unvorhersehbarer werden, werden zuerst veraltete oder weniger wichtige Aufgaben entfernt bzw. auf schnellere Varianten umgestellt.
4. **GPU-Kapazität wird nützlicher eingesetzt.** Rechenzeit, die sonst in bereits wertlose Resultate fließen würde, kann für aktuelle und höherwertige Inferenz verwendet werden.
5. **Hardware kann wirtschaftlicher genutzt werden.** Wenn ein Hersteller durch bessere Orchestrierung ein zusätzliches Modell auf vorhandener Hardware betreiben oder eine zusätzliche Compute-Einheit vermeiden kann, entsteht direkter Stückkostenhebel. Dies ist eine zu validierende wirtschaftliche Hypothese, keine garantierte Wirkung.

1. Problemdefinition in einfachen Worten

1.1 Warum normale Warteschlangen bei Robotern problematisch sind

Ein klassischer Server denkt in Requests: Ein Request kommt an, wird in die Queue gestellt und irgendwann abgearbeitet. Für viele Web- oder Business-Anwendungen ist das sinnvoll. Bei einem Roboter ist ein Teil der Arbeit jedoch **zeitlich verderblich**.

Beispiel: Eine Frontkamera liefert 30 Bilder pro Sekunde. Das bedeutet ein neues Bild etwa alle 33,3 ms. Unter Last benötigt die Objekterkennung plötzlich 50 ms pro Bild. Die Kamera erzeugt also 30 Aufgaben pro Sekunde, die GPU kann nur 20 davon pro Sekunde abarbeiten.

Bei FIFO entsteht eine Warteschlange:

Eingang: 30 Frames/s
Verarbeitung: 20 Frames/s
Differenz: 10 Frames/s Backlog

Nach zwei Sekunden liegen rechnerisch ungefähr 20 zusätzliche Frames in der Queue. Bei 50 ms Bearbeitungszeit je Frame entsprechen diese 20 Frames ungefähr **1 Sekunde Rückstand**. Der Detector kann dann technisch sehr schnell rechnen und trotzdem eine Welt analysieren, die für einen fahrenden oder greifenden Roboter bereits gefährlich alt ist.

Mit OneTimers LATEST-Semantik bleibt dagegen nur der aktuellste noch nicht gestartete Frame relevant:

```
Frame 101 running
Frame 102 queued
Frame 103 arrives -> 102 superseded
Frame 104 arrives -> 103 superseded
...
```

Der Detector schafft weiterhin nur etwa 20 Inferenzläufe pro Sekunde. OneTimer erfindet also keine zusätzliche GPU-Leistung. Aber er sorgt dafür, dass diese 20 Läufe auf möglichst aktuellen Daten stattfinden, statt einen immer älter werdenden Rückstand abzuarbeiten.

Das ist die grundlegende Produktidee: **nicht jede erzeugte Arbeit hat noch denselben Wert, wenn sie später ausgeführt wird.**

1.2 Die Garbage-Collector-Analogie

Die Analogie zu einem Garbage Collector ist konzeptionell nützlich, obwohl OneTimer keinen Speicher-Garbage-Collector implementiert.

Ein klassischer Garbage Collector erkennt Speicherobjekte, die keinen zukünftigen Nutzen mehr haben, und räumt sie auf. OneTimers **Stale Work Collector** erkennt Rechenaufgaben, deren Ergebnis aufgrund

neuerer Information keinen ausreichenden Nutzen mehr hat, und räumt diese aus der Warteschlange auf, bevor sie GPU-Zeit verbrauchen.

Speicher-GC: Objekt nicht mehr erreichbar -> freigeben
 OneTimer: Request nicht mehr nützlich -> nicht ausführen

Der Unterschied ist entscheidend: OneTimer verwaltet nicht primär Speicher, sondern **den zukünftigen Wert von Rechenarbeit**.

1.3 Warum das Problem mit VLM/LLM plus Detektor größer wird

Ein Edge-System kann beispielsweise gleichzeitig ausführen:

RF-DETR/YOLO Detector	30 Hz, Ziel 33 ms
Depth Model	15 Hz, Ziel 66 ms
Pose	30 Hz, Ziel 33 ms
Segmentation	10 Hz, Ziel 100 ms
VLM	ereignisbasiert, Ziel 500-1000 ms

Die periodischen Wahrnehmungsmodelle benötigen kurze und wiederkehrende GPU-Zeitfenster. Ein VLM kann dagegen deutlich längere GPU-Arbeit auslösen. Da GPU-Arbeit nicht wie ein normaler CPU-Thread beliebig und zuverlässig hart unterbrochen werden kann, muss die Steuerung **vor dem Start** einer langen Arbeit abschätzen, ob dadurch zukünftige wichtigere Arbeit gefährdet wird.

Aktuelle CUDA-Dokumentation beschreibt Stream-Prioritäten ausdrücklich als Hinweise; höher priorisierte Arbeit unterbricht bereits laufende, niedriger priorisierte Arbeit nicht zuverlässig [S5]. Genau deshalb ist OneTimer primär ein **Admission- und Work-Value-Governor**, nicht ein magischer GPU-Preemptor.

2. Produktthese und Marktthese

2.1 Was bereits existiert

Der Markt für Inference Serving existiert bereits. Das Produkt soll keinen neuen Markt schaffen. Zu den etablierten Bausteinen gehören insbesondere NVIDIA Triton, TensorRT, KServe/Open Inference Protocol und in Physical-AI-Stacks Holoscan. Triton ist Open Source unter BSD-3-Clause und bereits als Cloud- und Edge-Inference-Server etabliert [S1]. Das Open Inference Protocol definiert standardisierte REST- und gRPC-Schnittstellen und wird unter anderem von KServe, Triton, Seldon, OpenVINO und weiteren Runtimes verwendet [S2].

OneTimer setzt bewusst **oberhalb bzw. vor** diesem Marktstandard an. Der gewünschte Wechsel ist nicht:

"Wirf deinen Inferenzstack weg und lerne unser neues Framework."

sondern:

"Behalte Modelle und Triton; ändere den Endpoint und ergänze wenige QoS-Regeln."

2.2 Warum die technische Nachfrage plausibel ist

Die Nachfrage nach effizienter Edge-Inferenz ist öffentlich sichtbar. Figure sucht 2026 einen Staff AI Inference & Acceleration Engineer, der die Onboard-Inferenzarchitektur humanoider Roboter verantwortet und AI-Workloads über die Compute-Hardware hinsichtlich Latenz, Zuverlässigkeit, Leistung und Kosten abbildet [S10]. In Deutschland beschreibt Agile Robots/Idealworks Edge-Inferenz als zentrale Schicht ihres Robotik-Stacks und betont, durch Software- und Pipeline-Optimierungen mehr aus vorhandener Jetson-Hardware herauszuholen [S11]. NEURA sucht Perception-Expertise mit modernem C++/CUDA und Deployment auf NVIDIA Jetson [S12], ARX Robotics nennt unter anderem TensorRT/CUDA und Edge Computing für Echtzeit-Perception [S13].

Diese Signale beweisen **das technische Problem und vorhandenes Engineering-Budget**. Sie beweisen noch nicht, dass ein eigenständiger Markt für OneTimer groß genug ist. Das ist ausdrücklich Gegenstand der Produktvalidierung.

2.3 Deutschland als Design- und Startmarkt

Deutschland ist Europas größter Markt für industrielle Robotik. Laut IFR wurden 2024 in Deutschland 26.982 Industrieroboter installiert, 32 % des europäischen Jahresvolumens [S14]. Diese Gesamtzahl ist **kein direkter TAM für OneTimer**, denn nur ein Teil dieser Systeme betreibt mehrere konkurrierende Deep-Learning-Workloads auf einer gemeinsamen Edge-GPU. Sie zeigt aber, dass Deutschland als Robotik-, Automatisierungs- und Automotive-Standort genügend Design-Partner, Engineering-Kompetenz und potenzielle OEM-Anwendungen bietet.

Priorisierte Zielsegmente für frühe Validierung:

Segment	OneTimer-Fit	Begründung
Humanoide/mobile Robotik	sehr hoch	viele parallele Perception- und Reasoning-Modelle, knappe Edge-Hardware
Defense Robotics / UGV	sehr hoch	Edge-Betrieb, Sensorfusion, hohe Bedeutung frischer Wahrnehmung
Mobile Industrie-/Warehouse-Roboter	hoch	klare Edge-Plattformen, wirtschaftlicher Hardwarehebel
Drohnen	hoch	stark begrenzte Rechenleistung, Frische wichtiger als Backlog
Autonome Spezialfahrzeuge	hoch	mehrere KI-Pipelines, begrenzte Plattformen
Pkw Level 3/4	technisch hoch, Go-to-Market niedrig	Safety-, Qualifikations- und Lieferkettenhürden; erst später

3. Konkurrenzanalyse und reale Abgrenzung

3.1 NVIDIA Triton

Triton ist der wichtigste Referenz- und Kompatibilitätspartner. Es unterstützt konkurrierende Modellausführung, dynamisches Batching, Modellpipelines, HTTP/gRPC und Edge-Einsatz [S1]. Der Triton Rate Limiter kann über **alle geladenen Modelle hinweg** priorisieren und Ressourcen modellieren [S3]. Deshalb ist die Aussage "OneTimer hat Prioritäten, Triton nicht" falsch.

Triton besitzt außerdem Request Cancellation. Die aktuelle Dokumentation beschreibt jedoch Grenzen: Sobald Requests an bestimmte interne Stufen bzw. Backends weitergereicht wurden, ist frühe Beendigung nicht generell garantiert; backendseitige frühe Terminierung wird nur von bestimmten Backends unterstützt [S6]. Daraus folgt OneTimers Ansatz: **stale Arbeit möglichst vor dem Backend-Dispatch eliminieren**, statt darauf zu vertrauen, eine bereits laufende Inferenz zurückholen zu können.

OneTimer vs. Triton

Triton beantwortet primär:

Wie führe ich Modellinferenz effizient und skalierbar aus?

OneTimer beantwortet zusätzlich:

Ist dieser konkrete Request noch nützlich? Darf er jetzt überhaupt gestartet werden? Gefährdet er eine zeitkritische, erwartbare zukünftige Arbeit? Welche Qualitätsvariante ist noch rechtzeitig machbar?

3.2 NVIDIA Holoscan

Holoscan ist für OneTimer besonders relevant, weil es bereits Physical-AI- und Echtzeitmuster adressiert. Der aktuelle Holoscan Scheduler unterstützt unter anderem Event-basierte Ausführung; Async-Buffer besitzen eine "latest frame wins"-Semantik [S7]. Deshalb ist auch die Aussage "Latest Frame ist unser exklusiver USP" falsch.

Die aktuelle Holoscan-Inference-Dokumentation zeigt zugleich eine klare Angriffsfläche: Bei paralleler Multi-Model-Inferenz werden Modelle parallel gestartet, **ohne vorher zu prüfen, ob ausreichend GPU-Speicher und Compute-Ressourcen vorhanden sind**; der Nutzer muss dies selbst sicherstellen [S8]. Genau hier liegt OneTimers spezialisierter Mehrwert: globale, modellübergreifende Zulassungs- und Variantenentscheidung anhand gemessener Interferenz, Deadline und Freshness.

3.3 REEF und aktuelle Forschung

REEF ist ein real-time GPU DNN inference scheduling system mit Real-Time- und Best-Effort-Klassen und deutlich invasiveren Preemption-Techniken [S15]. Der Code ist Apache-2.0 lizenziert. REEF zeigt wissenschaftlich und technisch, dass die Scheduling-Schicht gegenüber einem generischen Serving-Stack relevante Vorteile erzeugen kann. Für OneTimer ist REEF jedoch **kein sinnvoller MVP-Unterbau**: die Preemption-Technik ist deutlich hardware- und runtime-näher, wartungsintensiver und nicht notwendig, um die erste Hypothese zu beweisen.

EdgeServing (2026) untersucht deadline-aware Multi-DNN-Serving auf einer gemeinsamen Edge-GPU und wählt systemweit Modell, Early Exit und Batchgröße anhand der Wirkung auf die zukünftige Queue-Situation [S9]. Auch dies bestätigt: Modellwahl und globale SLO-Wirkung sind reale Forschungsprobleme. OneTimer versucht nicht, diese Wissenschaft als neu zu beanspruchen, sondern sie **produktkompatibel, robust und installierbar** zu machen.

DISB ist ein Apache-2.0-lizenziertes Benchmark-Framework für DNN Inference Serving, das unter anderem Echtzeit-/Autonomous-Driving-artige Workloads abbildet [S16]. Für OneTimer sollte DISB als Benchmark-Baustein evaluiert und nach Möglichkeit adaptiert werden, statt eine komplette Benchmark-Infrastruktur unnötig neu zu schreiben.

3.4 Wettbewerbsmatrix

Fähigkeit	Triton	Holoscan	REEF/Forschung	OneTimer Ziel
Standardisiertes Inference-Protokoll	ja	nein/Framework	nein	ja
Multi-Model-Inferenz	ja	ja	ja	ja
Cross-Model-Priorität	ja	teilweise	ja	ja
Latest-frame-Semantik	nicht Kernfeature	ja	teilweise	ja, modellübergreifend integriert
Deadline-aware Admission vor Dispatch	begrenzt	nicht als Produktkern	Forschung	Kernfeature

Fähigkeit	Triton	Holoscan	REEF/Forschung	OneTimer Ziel
Freshness/Age-of-Information als First-Class-Semantik	nein	lokal über Buffer	Forschung	Kernfeature
Automatische Variantenwahl nach Feasibility	nein	nein	Forschung	Kernfeature
Interference-aware Zulassung	manuell konfigurierbar	Nutzer verantwortlich	Forschung	automatisiert
Drop-in vor vorhandenem Triton	n/a	nein	nein	ja
Produktziel: useful timely inference	nicht primär	nicht primär	ja	ja

3.5 Was OneTimer nicht behaupten darf

OneTimer darf im MVP nicht als "Hard-Realtime GPU Runtime" vermarktet werden. Es darf nicht behauptet werden, Triton besitze keine Priorisierung, Holoscan kenne keine Latest-Frame-Semantik oder NVIDIA könne keine parallele Inferenz. Die Differenzierung liegt ausschließlich in der **Kombination und Automatisierung**:

Freshness
 + deadline-aware admission
 + workload forecasting
 + model variants
 + interference profiles
 + standard protocol compatibility
 + graceful overload degradation

4. Kundennutzen mit Zahlen - Rechenbeispiele und Zielmetriken

4.1 Beispiel: Frische statt FIFO-Rückstand

Annahmen:

Kamera: 30 FPS -> 33,3 ms zwischen Frames
 Detector unter Last: 50 ms pro Inferenz -> 20 Inferences/s

FIFO erzeugt rechnerisch 10 Frames Backlog pro Sekunde. Nach 2 Sekunden sind ungefähr 20 Frames zusätzlich aufgelaufen. Bei 50 ms Bearbeitungszeit entspricht dies ungefähr 1 Sekunde zusätzlicher Queue-Zeit. Ein Roboter kann dann eine etwa eine Sekunde alte Szene verarbeiten.

Mit LATEST wächst die Warteschlange nicht entsprechend an. Ein laufender Request kann nicht magisch rückgängig gemacht werden, aber der **queued** Request wird jeweils durch den neuesten Frame ersetzt. Ergebnis: geringerer Information Age und gebundene Queue-Tiefe.

4.2 Beispiel: vermiedene stale GPU-Arbeit

Angenommen fünf Frames warten, und nur der jüngste ist für die betreffende Pipeline noch fachlich relevant. Wenn OneTimer vier der fünf queued Inferences vor dem Start entfernt, werden **bis zu 80 % dieser**

konkreten wartenden Rechenarbeit vermieden.

Das bedeutet ausdrücklich nicht automatisch 80 % weniger gesamte GPU-Last. Der globale Effekt hängt davon ab, welcher Anteil der Gesamtlast aus supersedierbarer Arbeit besteht.

Beispielszenario:

Gesamte GPU-Zeit, die ohne OneTimer stale wird:	20 %
Davon durch OneTimer vermeidbar:	75 %
Freigesetzte globale GPU-Zeit:	15 Prozentpunkte

15 Prozentpunkte zusätzliche nützliche Kapazität können je nach Stack genügen, um ein weiteres Modell zu betreiben, eine höherwertige Modellvariante häufiger einzusetzen oder mehr Lastspitzen zu absorbieren. Dies ist eine Validierungshypothese.

4.3 Beispiel: geschütztes Compute-Budget

Vereinfachte, vollständig serialisierte Betrachtung:

Detector:	10 ms alle 33 ms ->	ca. 30,3 % GPU-Budget
Depth:	12 ms alle 66 ms ->	ca. 18,2 %
Pose:	8 ms alle 33 ms ->	ca. 24,2 %

 Protected total -> ca. 72,7 %
 Theoretischer serieller Rest-> ca. 27,3 %

Ein Scheduler kann daraus keine 120 % reale Kapazität machen. Er kann aber erkennen, dass ein Best-Effort-VLM, das ein langes unteilbares Ausführungsfenster benötigt, **nicht beliebig jederzeit gestartet werden darf**, wenn in Kürze geschützte periodische Arbeit erwartet wird.

4.4 Produkt-Go/No-Go-Zielwerte

Die erste technische Validierung gilt nur dann als erfolgreich, wenn OneTimer gegenüber einer **gut konfigurierten Triton-Baseline** mindestens einen großen Effekt zeigt:

- **Ziel A:** mindestens 2x weniger Protected/Critical Deadline Misses unter relevanter Überlast; oder
- **Ziel B:** mindestens 30 % weniger GPU-Zeit, die auf bei Fertigstellung bereits obsoleter Inferenz entfällt;
- **Ideal:** beide Effekte gleichzeitig.

Zusätzliche Zielgrenzen:

- Normalbetrieb: weniger als 3-5 % End-to-End-Performance-Regressions gegenüber direktem Triton im vergleichbaren Transportmodus.
- Scheduler-Entscheidung selbst: p99 unter 100 Mikrosekunden auf moderner x86-Hardware als Entwicklungsziel.
- Keine unbegrenzte Queue und kein lineares Speicherwachstum unter Überlast.
- Kein stiller Verlust von NEVER_DROP-Requests.

Diese Werte sind **Produkt-Gates**, keine bereits erreichten Benchmarks.

4.5 Wirtschaftlicher Effekt - Szenarien

Der wirtschaftliche Wert sollte nicht nur als eingesparte Entwicklerzeit argumentiert werden. Der stärkere Hebel kann Hardware und Funktionsdichte sein.

Beispielhafte Formel:

Wert = vermiedene zusätzliche Compute-BOM
 + vermiedener eigener Scheduling-/Tuning-Aufwand
 + Wert zusätzlicher Modelle/Funktionen auf bestehender Hardware
 + geringeres Risiko unkontrollierter Überlast

Illustratives OEM-Szenario, ausdrücklich keine Marktprognose:

Zusätzliche Compute-Einheit, die sonst nötig wäre: 800 EUR

Ausgelieferte Systeme: 5.000

Maximaler BOM-Hebel, falls vollständig vermieden: 4,0 Mio. EUR

Falls nur bei 20 % der Systeme vermeidbar: 0,8 Mio. EUR

Das zeigt, warum selbst eine relativ teure B2B-/OEM-Lizenz wirtschaftlich vertretbar sein kann, **wenn** OneTimer nachweisbar eine Hardwarestufe, einen zweiten Edge-Rechner oder einen Funktionsverzicht vermeidet. Genau diese Hypothese muss später mit echten OEM-Stacks validiert werden.

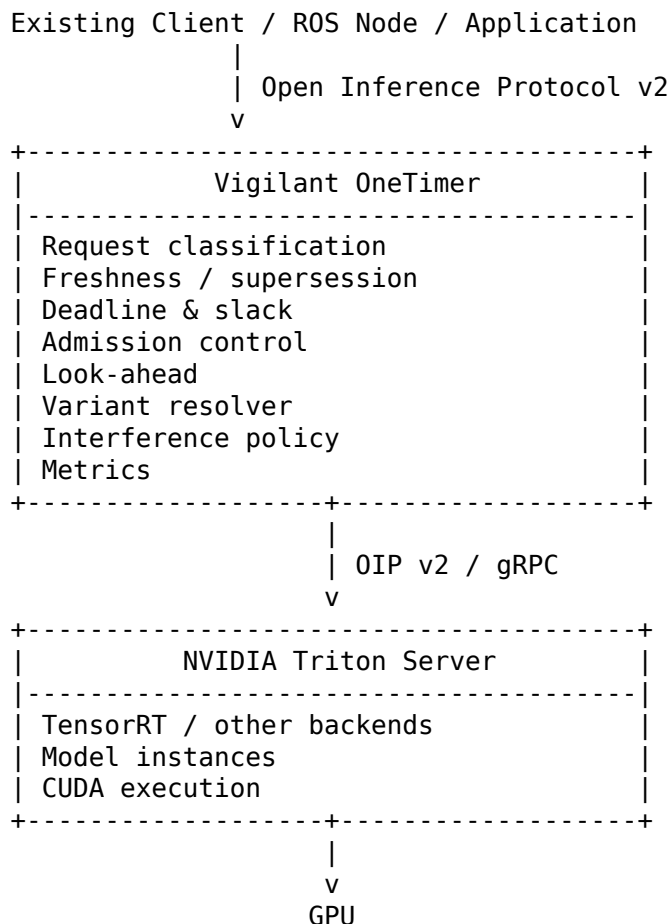
5. Produktarchitektur - endgültige MVP-Entscheidung

5.1 OneTimer wird Triton zunächst nicht neu schreiben

Die wichtigste Vereinfachung für den MVP lautet:

OneTimer wird als kompatibler Governor/Gateway vor Triton gebaut, nicht als neuer TensorRT-Inference-Server von Grund auf.

Architektur:



Vorteile:

1. Kein eigener TensorRT-Server im MVP.
2. Keine eigene Modell-Repository-Implementierung nötig.
3. Bestehende Triton-Backends bleiben nutzbar.
4. Open Inference Protocol ist bereits standardisiert und Apache-2.0 lizenziert [S2].

5. Triton ist BSD-3-Clause lizenziert [S1].
6. OneTimer kann zunächst als vollständig separate Open-Source-Komponente gebaut werden.
7. Der Kernnutzen kann isoliert benchmarked werden, bevor tiefer in CUDA eingegriffen wird.

5.2 Warum der MVP-Core in Rust gebaut werden sollte

Für diese Gateway-/Governor-Architektur ist Rust gegenüber einem sofortigen C++/CUDA-Core vorteilhaft:

- kein Garbage Collector und damit keine GC-Pausen;
- Memory Safety für eine dauerhaft laufende Infrastrukturkomponente;
- gute gRPC-/async-Unterstützung;
- niedriger Overhead;
- einfacher bounded-memory-orientierter Entwurf;
- gute Eignung für einen Coding-Agenten mit strengem Compilerfeedback;
- keine direkte CUDA-Abhängigkeit im ersten Produktkern.

Empfehlung:

MVP Governor: Rust stable

Async runtime: Tokio

OIP gRPC: tonic/prost

Konfiguration: serde + YAML

Observability: tracing + Prometheus-compatible metrics

CLI: clap

Ein späterer nativer TensorRT-/CUDA-Executor kann als C++-Modul oder separater Backend-Prozess ergänzt werden, wenn der Gateway-Ansatz nachweislich an eine technische Grenze stößt.

5.3 Data Path und Control Path trennen

OneTimer soll fachlich Requests steuern, aber nicht unnötig große Tensoren kopieren.

Phase 1:

Client -> localhost gRPC -> OneTimer -> localhost gRPC -> Triton

Dies ist die einfachste Kompatibilitätsstufe und reicht für Scheduling-Validierung.

Phase 1b:

Tritons Shared-Memory-Extension nutzen. Triton dokumentiert, dass system bzw. CUDA shared memory den Datentransfer für manche Workloads deutlich beschleunigen kann; auf Jetson wird aktuell nur **system shared memory** unterstützt [S4]. OneTimer muss deshalb Shared-Memory-Referenzen möglichst transparent weiterreichen bzw. verwalten, statt Tensorpayloads unnötig zu kopieren.

Langfristig:

- x86/dGPU: CUDA shared memory / IPC prüfen.
- Jetson: system shared memory und gegebenenfalls In-Process-/C-API-Bridge evaluieren.
- Ein nativer Executor ist erst gerechtfertigt, wenn der Proxy-Overhead oder die mangelnde Kontrolle über bereits weitergereichte Arbeit den Produktnutzen begrenzt.

6. Plug-and-Play-Integration

6.1 Zielintegration

Vorher:

client -> triton:8001

Nachher:

```
client -> onetimer:9001 -> triton:8001
```

Bei bestehenden Clients ohne OneTimer-spezifische Parameter arbeitet OneTimer im **transparenten Compatibility Mode**:

- physischer Modellname bleibt bestehen;
- Standard-OIP-Request bleibt gültig;
- Ankunftszeit wird als Fallback für Generation Time verwendet;
- Queue- und Deadline-Regeln kommen aus YAML-Defaults;
- unbekannte OIP-Felder dürfen nicht unnötig zerstört werden.

Für volle Freshness-Semantik können optionale Parameter ergänzt werden:

```
onetimer_generation_ns  
onetimer_stream_id  
onetimer_supersession_key  
onetimer_deadline_us  
onetimer_max_age_us
```

6.2 Beispielkonfiguration

```
version: 1  
backend:  
  type: triton  
  grpc_endpoint: "127.0.0.1:8001"  
  
models:  
  detector:  
    backend_model: detector_large  
    class: protected  
    queue:  
      policy: latest  
      key: stream_id  
      capacity: 1  
    contract:  
      period_ms: 33  
      deadline_ms: 30  
      max_age_ms: 66  
    variants:  
      - id: large  
        backend_model: detector_large  
        quality: 1.00  
      - id: small  
        backend_model: detector_small  
        quality: 0.93  
  
  vlm:  
    backend_model: vlm_main  
    class: best_effort  
    queue:  
      policy: fifo  
      capacity: 4  
    contract:  
      deadline_ms: 800  
      max_age_ms: 1500
```

6.3 Minimaler Nutzerworkflow

1. Offizielles Triton/NVIDIA-Image separat beziehen und starten

2. OneTimer installieren

```
oneter doctor -c oneter.yaml
```

3. Modelle profilieren

```
oneter profile -c oneter.yaml
```

4. Governor starten

```
oneter serve -c oneter.yaml
```

5. Im Client nur Zielpoint auf OneTimer ändern

6.4 Docker-Quickstart

Das Repository darf einen docker-compose.yml-Beispielstack anbieten, sollte das proprietär lizenzierte NVIDIA-Containerartefakt jedoch nicht ungeprüft selbst redistributieren. Das Compose-File referenziert das offizielle NVIDIA-Image; der Nutzer muss es aus der offiziellen Registry beziehen und den NVIDIA-Bedingungen zustimmen.

7. Funktionale Produktanforderungen - Lastenheft

7.1 Muss-Anforderungen MVP

L-001 - OIP-Kompatibilität: OneTimer muss die für den MVP benötigten Open-Inference-Protocol-v2-gRPC-Endpunkte so implementieren, dass ein Standardclient ohne kundenspezifisches SDK inferieren kann.

L-002 - Transparenter Proxy-Modus: Ein Modell ohne OneTimer-Sonderkonfiguration muss durchgeleitet werden können.

L-003 - Bounded Queues: Jede Queue besitzt eine explizite maximale Kapazität. Es darf keine unbegrenzte Warteschlange geben.

L-004 - Queue Policy LATEST: Bei supersedierbaren Streams darf höchstens der jüngste noch nicht gestartete Request erhalten bleiben.

L-005 - Queue Policy LATEST_PER_KEY: Pro Key muss jeweils nur die aktuellste queued Anfrage erhalten bleiben können.

L-006 - Queue Policy FIFO: Sequenzielle, nicht supersedierbare Requests müssen in Reihenfolge verarbeitet werden.

L-007 - Queue Policy NEVER_DROP: Requests dürfen nicht durch Freshness-/Supersession-Regeln verworfen werden. Dies ist keine Garantie, dass sie jede Deadline einhalten.

L-008 - Generation Time: OneTimer muss zwischen Erzeugungs-/Capture-Zeit und Arrival Time unterscheiden können.

L-009 - Max Age: Ein Request darf vor Dispatch als stale verworfen werden, wenn sein fachliches Alter die konfigurierte Grenze überschritten hat.

L-010 - Deadline: Pro Request bzw. Modellvertrag muss eine absolute Deadline aus Generation Time + relativer Deadline ermittelt werden können.

L-011 - Criticality: Mindestens PROTECTED, HIGH, NORMAL, BEST Effort.

L-012 - Admission Control: OneTimer darf einen Request ablehnen, warten lassen oder degradieren, wenn die konservative Planung ergibt, dass er nicht mehr sinnvoll ausführbar ist oder geschützte zukünftige Arbeit gefährdet.

L-013 - Variant Selection: Ein logisches Modell darf mehrere physische Backendmodelle besitzen. OneTimer wählt die qualitativ höchste konservativ machbare Variante.

L-014 - Runtime Profile: OneTimer muss p50/p95/p99-ähnliche Laufzeitprofile und Umgebungsfingerprints verwalten.

L-015 - Interference Profile: Relevante Modellpaare müssen offline einzeln und gemeinsam gemessen werden können.

L-016 - Overload States: Überlast muss zu kontrollierter Degradation statt unbeschränktem Queue-Wachstum führen.

L-017 - Metrics: Deadline Misses, Age of Information, stale drops, obsolete completions, Queuezeiten, Variantenwahl und Scheduler-Latenz müssen messbar sein.

L-018 - Determinismus: Schedulinglogik muss ohne LLM oder RL auf dem Hot Path funktionieren und bei identischem Simulationszustand reproduzierbar sein.

L-019 - Monotonic Time: Scheduling darf nicht auf einer durch NTP/Wall-Clock korrigierbaren Uhr basieren.

L-020 - Fail-safe Configuration: Ungültige Konfiguration darf den Prozess nicht stillschweigend mit riskanten Defaultwerten starten lassen.

7.2 Soll-Anforderungen nach MVP

- HTTP/OIP zusätzlich zu gRPC.
- Shared-Memory-Passthrough/Management.
- Stateful Sequence Handling.
- Pipeline/DAG-End-to-End-Deadlines.
- VLM/LLM-cooperative execution adapter.
- Multi-Accelerator und DLA/NPU.
- Native TensorRT-Ausführung optional.
- Fleet Profiles, Enterprise Policy und signed configs.

8. Nichtfunktionale Anforderungen

8.1 Performance

- Schedulerentscheidung p99 Ziel <100 us auf x86 im synthetischen Hot-Path-Test.
- Keine Heap-Allokationen im eigentlichen Scheduling-Entscheidungspfad, soweit praktisch erreichbar.
- Alle Queues bounded.
- Kein $O(n^2)$ -Verhalten in der Onlineplanung bei üblicher MVP-Konfiguration; Ziel $N \leq 32$ aktive logische Queues.
- Langsame Profilierungs-/Optimierungsarbeit offline durchführen; online nur Lookup und kurze Simulation.

8.2 Stabilität

- Dauerstresstest mit mindestens 10 Mio. synthetischen Requestzustandsübergängen ohne Deadlock und ungebundenes Speicherwachstum.
- Crash eines Backend-Requests darf den Scheduler nicht blockieren.
- Backend-Verbindungsverlust muss zu definierten Fehlerzuständen führen.
- Jeder Request besitzt einen expliziten terminalen Zustand.

8.3 Sicherheit

- Größenlimits für Requests und Tensor-Metadaten.

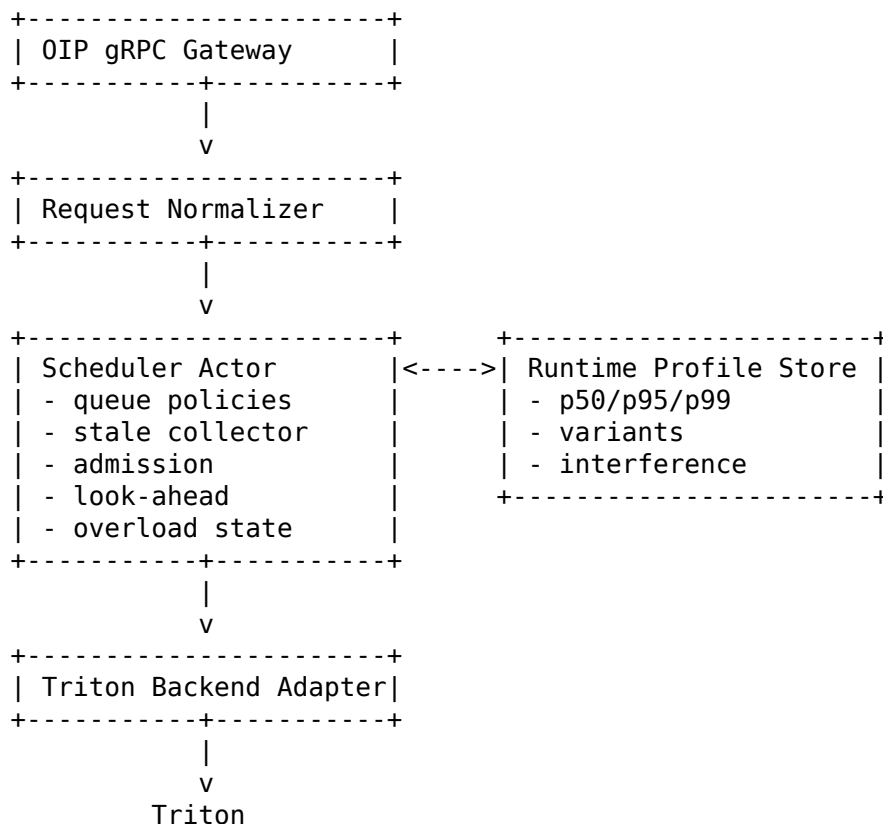
- gRPC/HTTP-Parser darf keine unbounded allocations aus fremd kontrollierten Größen erzeugen.
- Konfiguration schema-validieren.
- Keine Shell-Ausführung aus Modellnamen oder Clientparametern.
- Optional TLS/mTLS im Enterprise-Ausbau.

8.4 Wartbarkeit

- Scheduling-Core ohne Triton-Abhängigkeit testbar.
- Backend-Adapter abstrahiert.
- Protokoll-Adapter abstrahiert.
- Jede Schedulingregel mit deterministischen Golden Tests.
- Dokumentierte Compatibility Matrix.

9. Pflichtenheft - interne Systemarchitektur

9.1 Komponenten



Zusätzliche Nebenkompenten:

Config Loader / Validator
 Profiler
 Benchmark Adapter
 Metrics Exporter
 Audit/Event Logger
 Health/Readiness

9.2 Repository-Struktur

```
onetimer/
├── Cargo.toml
├── Cargo.lock
├── LICENSE
├── NOTICE
├── THIRD_PARTY_NOTICES.md
├── SECURITY.md
├── CONTRIBUTING.md
├── README.md
├── deny.toml
├── .reuse/
├── crates/
│   ├── onetimer-core/           # reine Schedulinglogik
│   ├── onetimer-config/        # Schema, Parser, Validator
│   ├── onetimer-protocol-oip/  # OIP v2 DTO/Mapping
│   ├── onetimer-gateway/       # gRPC/HTTP Server
│   ├── onetimer-backend-triton/ # Backend-Adapter
│   ├── onetimer-profiler/      # Runtime-/Interferenzprofile
│   ├── onetimer-metrics/       # Metrics + structured events
│   └── onetimer-cli/           # doctor/profile/serve/bench
├── proto/
│   └── oip/
├── tools/
│   ├── benchmark/
│   ├── triton-baseline/
│   ├── workload-generator/
│   └── license-report/
├── tests/
│   ├── property/
│   ├── golden/
│   ├── compatibility/
│   ├── integration/
│   ├── stress/
│   └── fuzz/
├── deploy/
│   ├── docker/
│   └── docker-compose/
├── examples/
│   ├── detector_latest/
│   ├── detector_plus_vlm/
│   └── multi_model_overload/
└── docs/
    ├── architecture/
    ├── protocol/
    ├── benchmark/
    ├── licensing/
    └── adr/
```

9.3 Kern-Datentypen

```
pub enum QueuePolicy {
    Latest,
    LatestPerKey,
    Fifo,
    NeverDrop,
}

pub enum Criticality {
    Protected,
    High,
    Normal,
    BestEffort,
}

pub enum RequestState {
    Received,
    Queued,
    Superseded,
    Stale,
    RejectedInfeasible,
    Admitted,
    Forwarded,
    CompletedValid,
    CompletedObsolete,
    Failed,
}
```

Ein Requestdescriptor trennt kleine Scheduling-Metadaten von großen Tensorpayloads:

```
pub struct RequestDescriptor {
    pub id: RequestId,
    pub logical_model: ModelId,
    pub stream_id: Option<StreamId>,
    pub supersession_key: Option<SupersessionKey>,

    pub generation_time: MonotonicInstant,
    pub arrival_time: MonotonicInstant,
    pub absolute_deadline: Option<MonotonicInstant>,
    pub max_age: Option<Duration>,

    pub criticality: Criticality,
    pub queue_policy: QueuePolicy,
    pub stateful: bool,

    // Referenz auf Payload; nicht notwendigerweise Besitz/Kopie.
    pub payload_ref: PayloadRef,
}
```

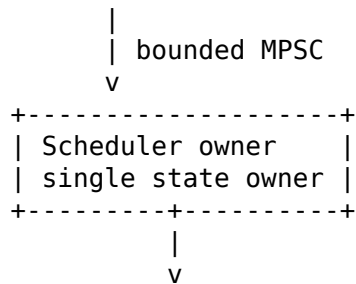
9.4 Single-owner Scheduler Actor

Der Schedulerzustand soll im MVP von genau einem logischen Actor/Task besessen werden. Netzwerkworker senden Ereignisse über bounded channels an diesen Actor. Vorteile:

- keine komplexen Locks im eigentlichen Scheduler;
- deterministische Zustandsübergänge;

- einfache Replay-Tests;
- bessere Analyse bei Race Conditions;
- klare Backpressure.

Network workers



Backend dispatch tasks

Backend-I/O darf asynchron sein, aber eine erfolgreiche oder fehlgeschlagene Backend-Aktion wird als Event zurück an den Scheduler gemeldet.

10. Tiefe Scheduling-Theorie

10.1 Zielgröße: Useful Timely Inference

Server-Benchmarks maximieren häufig Throughput oder minimieren durchschnittliche Latenz. OneTimer verwendet als Produktintuition:

Useful Timely Inference =

Ergebnisse, die fachlich noch aktuell und innerhalb ihres Vertrages ankommen

verwendete Compute-Ressource

Es handelt sich zunächst um eine Produktmetrik, nicht um eine einzige universelle mathematische Zielfunktion. Für den Scheduler werden mehrere Ziele lexikographisch priorisiert.

10.2 Latency ist nicht Age of Information

Vier Zeitpunkte müssen getrennt werden:

t_g = generation/capture time

t_a = arrival at OneTimer

t_d = dispatch to backend

t_c = completion/delivery

Daraus:

Queue Delay = $t_d - t_a$

Compute/Backend = $t_c - t_d$

Request Latency = $t_c - t_a$

Information Age = $t_c - t_g$

Ein Frame kann geringe Request Latency und trotzdem hohe Information Age besitzen, wenn er bereits vor dem Eintreffen im Gateway alt war. Deshalb wird die absolute Deadline aus `generation_time` abgeleitet, sofern der Client diese liefert.

10.3 Drei Stufen der Stale-Pruefung

Stufe A - Ingress Supersession

Sobald ein neuer Request eintrifft, werden überholte queued Requests gemäß Policy entfernt.

Stufe B - Pre-dispatch Freshness

Direkt vor einer geplanten Backend-Ausführung wird erneut geprüft:

`now - generation_time <= max_age`

und:

`predicted_finish <= absolute_deadline`

Ist das nicht erfüllt, darf ein supersedierbarer Request verworfen werden.

Stufe C - Completion Validity

Nach der Backendantwort wird geprüft, ob das Resultat bei Fertigstellung noch aktuell genug ist. Ein COMPLETED_OBSOLETE-Resultat kann je nach Konfiguration nicht an den Client geliefert oder mit einem Metadatenstatus versehen werden. Die GPU-Zeit ist zu diesem Zeitpunkt bereits verbraucht und wird als `stale_compute_ms` erfasst.

10.4 Slack / Laxity

Für einen Request j und eine Kandidatenvariante v :

$\text{slack}(j,v) = \text{deadline}(j) - \text{now} - \text{predicted_runtime}(j,v)$

Interpretation:

- großer positiver Slack: Request kann warten;
- kleiner positiver Slack: zeitkritisch;
- negativer Slack: mit dieser Variante nicht mehr rechtzeitig machbar.

Bei mehreren Varianten wird der Slack pro Variante berechnet.

10.5 Earliest Deadline First als Basis, nicht als Endloesung

EDF priorisiert die kleinste absolute Deadline. Für vollständig präemptive Single-Processor-Modelle besitzt EDF starke theoretische Eigenschaften. Eine Edge-GPU mit langen nicht hart präemptierbaren Inferenzabschnitten entspricht diesem Modell jedoch nicht sauber. Deshalb kombiniert OneTimer EDF mit:

- Criticality;
- Slack;
- Non-preemptive blocking awareness;
- Future-arrival look-ahead;
- Variant selection;
- Freshness;
- Interference profiles.

10.6 Lexikographische Zielreihenfolge

OneTimer soll keine einzige gewichtete "Magic Score"-Funktion verwenden, die einen Protected-Request gegen viele Best-Effort-Requests verrechnen kann.

MVP-Zielreihenfolge:

1. Protected Deadline Misses minimieren.
2. High Deadline Misses minimieren.
3. Stale/obsolete Compute minimieren bzw. Informationsfrische maximieren.
4. Qualität innerhalb der machbaren Varianten maximieren.
5. Best-Effort-Durchsatz/GPU-Auslastung maximieren.

Das System darf deshalb bewusst einen niedrig priorisierten Request nicht starten, obwohl die GPU gerade frei erscheint.

10.7 Warum absichtliches Idle korrekt sein kann

Ein klassischer Work-Conserving Scheduler versucht, eine freie Ressource möglichst nie absichtlich ungenutzt zu lassen. Für eine nicht hart präemptierbare GPU kann dies falsch sein.

Beispiel:

Jetzt: $t = 0$ ms
 Best-Effort VLM Laufzeit: 50 ms
 Protected Detector erwartet: $t = 8$ ms
 Detector Deadline nach Ankunft: 20 ms

Wenn der VLM-Job die relevanten GPU-Ressourcen so belegt, dass der Detector erst sehr spät starten kann, wäre es besser, wenige Millisekunden bewusst freizuhalten. OneTimer darf daher **non-work-conserving** entscheiden:

GPU idle 0-8 ms
 Detector 8-20 ms
 VLM danach

Diese Entscheidung ist ein Kernunterschied zu einem reinen "immer sofort möglichst viel ausführen"-Ansatz.

10.8 Periodische Future Arrivals

Bei Physical AI sind viele Requests periodisch. Konfiguration:

period_ms: 33

OneTimer hält für geschützte periodische Modelle einen erwarteten nächsten Arrival-Zeitpunkt. Dies ist keine Garantie, dass ein Frame exakt dann eintrifft; es ist eine Scheduling-Prognose.

Ein Look-ahead von beispielsweise 100 ms kann vorhersehbare zukünftige Protected-Requests berücksichtigen.

10.9 Demand-/Utilization-Analyse offline

Für einen vereinfachten serialisierten Protected-Workload mit Tasks

$\tau_i = (C_i, T_i, D_i)$

wobei:

- C_i : konservative Ausführungszeit;
- T_i : Periode;
- D_i : relative Deadline,

ist eine einfache notwendige Warnmetrik:

$U = \sum (C_i / T_i)$

Wenn $U > 1$, kann ein einzelner vollständig serialisierter Executor die Last offensichtlich nicht dauerhaft tragen. Bei realer GPU-Konkurrenz, Parallelität und nicht-präemptiven Abschnitten ist diese Formel keine vollständige Schedulability-Garantie. Sie ist aber extrem nützlich, um bereits beim onetimer doctor unsinnige Verträge zu erkennen.

OneTimer soll bei offensichtlich unmöglicher Protected-Konfiguration nicht optimistisch starten, sondern z.B.:

PROTECTED_WORKLOAD_UNSCEDULABLE

melden und konkrete Kandidaten für Degradation/Variantenwechsel zeigen.

10.10 Non-preemptive Blocking

Da eine bereits laufende GPU-Arbeit nicht zuverlässig hart unterbrochen werden kann [S5], muss das System ein konservatives Blocking-Modell pflegen.

MVP:

```
blocking_estimate(variant) = conservative end-to-end backend runtime
```

Dies ist grob, aber sicherer als eine unrealistische Preemption-Annahme.

Vollausbau:

- CUPTI/Nsight-basierte Messung langer Kernelabschnitte;
- max_nonpreemptible_segment_us je Variante;
- kooperative Yield Points für generative Modelle;
- optional experimenteller Preemption-Backend-Ansatz.

10.11 Protected Slack Server

Best-Effort-Arbeit darf nur in Kapazität ausgeführt werden, die nach konservativer Look-ahead-Simulation verbleibt.

Algorithmische Intuition:

```
reserve enough capacity for known protected horizon  
use remaining slack for normal/best-effort work
```

Damit wird ein Best-Effort-VLM zum "Slack Consumer" statt zum unkontrollierten Konkurrenzmodell.

11. Stale Work Collector im Detail

11.1 LATEST

Invariante:

Anzahl queued, nicht laufender Requests pro LATEST-Key ≤ 1

Ein bereits FORWARDEDer Request wird im MVP nicht als zuverlässig abbrechbar angenommen. Ein neuer Request ersetzt daher nur queued Arbeit.

11.2 LATEST_PER_KEY

Anwendungsfälle:

- mehrere Kameras;
- Tracking je Objekt-ID;
- unterschiedliche Sensorstreams;
- unterschiedliche Kunden/Robotersubsysteme.

Invariante:

queued(key) ≤ 1

11.3 FIFO

Geeignet für:

- einmalige Nutzerkommandos;
- bestimmte Sprach-/Agentenanfragen;
- nicht supersedierbare Inferenzsequenzen.

FIFO benötigt eine explizite Kapazität und ein Overflow-Verhalten:

reject_new
reject_oldest_non_protected
backpressure_client

Kein stilles unbounded buffering.

11.4 NEVER_DROP

NEVER_DROP bedeutet:

- nicht wegen eines neueren Requests supersedieren;
- bei voller Queue explizite Backpressure/Fehlermeldung statt stillem Drop;
- alle terminalen Zustände auditierbar.

Es bedeutet **nicht**:

- hard-real-time guarantee;
- garantierte erfolgreiche Backendausführung;
- garantierte unendliche Pufferung.

11.5 SAMPLE / DECIMATE - spaeter

Optional kann eine Pipeline künftig definieren:

process at most every Nth frame

oder:

target_effective_rate_hz

Dies darf im MVP nicht die zentrale Logik verkomplizieren. LATEST plus Backpressure liefert bereits den Kernnutzen.

12. Model Variant Selection

12.1 Logisches vs. physisches Modell

Die Anwendung fragt:

detector

OneTimer kann weiterleiten an:

detector_large
detector_medium
detector_small

Alle Varianten müssen semantisch kompatible Outputs besitzen oder über einen expliziten Adapter normalisiert werden.

12.2 Qualitätswert im MVP

OneTimer darf Qualität nicht automatisch erfinden. Jede Variante erhält einen durch den Nutzer oder Benchmark ermittelten relativen Score:

`quality: 1.00`

oder zusätzlich:

`min_acceptable_quality: 0.92`

12.3 Auswahlregel

Sortiere Varianten absteigend nach Qualität. Wähle die erste, die nach konservativer Look-ahead-Simulation feasible ist.

```
Large predicted finish 38 ms, deadline 33 -> nein
Medium predicted finish 25 ms, deadline 33 -> ja
Small predicted finish 18 ms, deadline 33 -> ja
```

=> Medium

12.4 Hysterese gegen Modell-Flattern

Ständiges Wechseln Large/Small/Large kann unerwünschte zeitliche Instabilität und unterschiedliche Outputs erzeugen. Deshalb benötigt der Voll-MVP mindestens optional:

`min_variant_dwell_ms`
`upgrade_hysteresis`

Prinzip:

- Abwertung auf kleinere Variante darf schnell erfolgen, wenn Feasibility dies verlangt.
- Aufwertung auf größere Variante erst nach stabiler Reserve über mehrere Fenster.

12.5 Stateful Models

Stateful/sequence-basierte Modelle dürfen nicht blind zwischen Varianten wechseln. Konfiguration:

`stateful: true`
`variant_policy: pinned_for_sequence`
`queue_policy: fifo`

Triton besitzt bereits Sequence-Batching-/Stateful-Mechanismen; OneTimer muss diese Semantik respektieren, statt sie zu beschädigen [S17].

13. Runtime Profiling und Interferenz

13.1 Warum Durchschnittslatenz nicht reicht

Der Scheduler benötigt konservative Laufzeitannahmen. Deshalb werden mindestens p50, p95 und p99 erfasst.

Profil:

`model_variant`
`hardware_fingerprint`
`input_shape`
`p50_us`
`p95_us`

p99_us
sample_count
profile_timestamp

13.2 Online Runtime Estimator

Online werden tatsächliche Backendzeiten gemessen. Der konservative Schätzer kann initial sein:

$\text{predicted} = \max(\text{offline_p99}, \text{online_p99}) * \text{safety_margin}$

Safety Margin z.B. initial 1,10, mit hart begrenztem Bereich.

Wichtig: OneTimer benötigt dafür im MVP **keine direkte Temperaturmessung**. Wenn die Hardware aus irgendeinem Grund langsamer wird, steigt die gemessene Online-Laufzeit und der Scheduler reagiert auf den Effekt statt auf die Ursache.

13.3 Langsame Margin-Anpassung

Regeln:

- Nach Deadline-Miss oder deutlicher Runtime-Unterprognose Margin schrittweise erhöhen.
- Bei langer stabiler Phase Margin sehr langsam reduzieren.
- Keine aggressive Oszillation.
- Min/Max-Grenzen.

13.4 Interference Matrix

Modelle beeinflussen einander auf einer GPU. Deshalb offline messen:

A allein
B allein
A+B parallel

Beispiel:

Paar	A slowdown	B slowdown	Entscheidung
Detector + Depth	1,12x	1,08x	parallel meist erlaubt
Detector + VLM	2,80x	1,35x	geschuetzt kritisch
Segmentation + VLM	1,70x	1,22x	workloadabhaengig

Online wird kein teures Profiling durchgeführt. Der Scheduler benutzt nur gespeicherte Faktoren.

13.5 Profile Fingerprint

Ein Profil muss an eine Umgebung gebunden sein:

GPU/SoC identification
Triton version
Backend version
TensorRT version, soweit ermittelbar
model/version hash
input shape/dtype
OneTimer profiler version

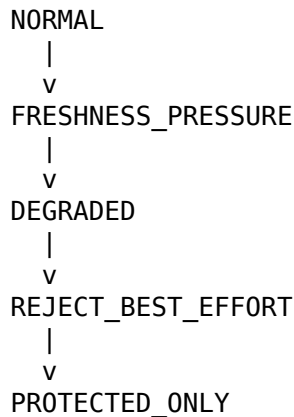
Bei relevanter Abweichung:

PROFILE_STALE

und konservativer Fallback oder erneutes Profiling.

14. Overload Control und Graceful Degradation

14.1 Zustandsmaschine



Ein Zustand wird nicht anhand einer einzelnen Momentmessung gewechselt, sondern über gleitende Fenster und Hysterese.

14.2 Beispielaktionen

NORMAL

- höchste feasible Qualität;
- Best-Effort erlaubt;
- normale Queue-Grenzen.

FRESHNESS_PRESSURE

- aggressiveres Superseding queued LATEST-Arbeit;
- keine unnötige Vorhaltung älterer Frames;
- Variantenaufwertung konservativer.

DEGRADED

- kleinere Modellvarianten zulässig/erzwungen;
- Normal/Best-Effort niedriger gewichten.

REJECT_BEST_EFFORT

- keine neue Best-Effort-Arbeit annehmen, wenn Protected-Horizont unter Druck steht.

PROTECTED_ONLY

- nur Protected/High gemäß Konfiguration;
- explizite Metrik und Alarm.

14.3 Hysterese

Wenn Eintritt in DEGRADED z.B. bei 10 % Protected Deadline Miss Pressure erfolgt, darf Rückkehr nicht bereits bei 9,9 % stattfinden. Rückkehrschwellen müssen deutlich niedriger und über ein Mindestfenster stabil sein.

Ziel: kein Zustandspendeln, kein Variantenflattern.

15. VLM/LLM-Sonderfall

15.1 Problem

Ein langer generativer Workload kann einen erheblichen Teil der GPU über lange Zeit beanspruchen. Da OneTimer im MVP keine zuverlässige harte GPU-Preemption besitzt, darf kein unrealistisches Versprechen gegeben werden.

15.2 MVP-Verhalten

- VLM/LLM zunächst BEST_EFFORT oder explizit NORMAL.
- Admission nur, wenn Protected-Look-ahead ausreichend Slack zeigt.
- Konservative Laufzeitprofile.
- Falls Backend-Cancellation unterstützt wird, kann dies als Optimierung genutzt werden, aber nicht als Safety-Grundlage [S6].

15.3 Vollausbau: kooperative Quanten

Generative Modelle bieten natürliche Yield Points:

- Token-Decoding;
- chunked prefill;
- iterative Agenten-/Reasoning-Schritte;
- Early Exit.

Ein späterer TensorRT-LLM/vLLM-spezifischer Adapter kann Best-Effort-Generation in kontrollierbare Quanten zerlegen, damit Protected-Arbeit zwischen Quanten bevorzugt wird.

16. Protokoll und API

16.1 Open Inference Protocol v2

OneTimer implementiert mindestens:

- Server Live;
- Server Ready;
- Model Ready;
- Model Metadata;
- Model Infer.

OIP definiert standardisierte REST- und gRPC-Schnittstellen und ist Apache-2.0 lizenziert [S2].

16.2 OneTimer Parameter

Alle Erweiterungen verwenden einen klaren Prefix:

```
onetimer_generation_ns  
onetimer_stream_id  
onetimer_supersession_key  
onetimer_deadline_us  
onetimer_max_age_us  
onetimer_class
```

Der Server muss unbekannte Standardparameter möglichst transparent behandeln bzw. dokumentieren, wenn sie nicht unterstützt werden.

16.3 Compatibility Mode

Wenn kein OneTimer-Parameter mitgeliefert wird:

1. Modellkonfiguration wird anhand des logischen/physikalischen Namens gesucht.
2. `generation_time` = `arrival_time` als Fallback.
3. Contract/Policy aus Serverconfig.
4. Falls keine OneTimer-Config vorhanden: transparentes Forwarding.

Dies ermöglicht eine sehr niedrige Integrationshürde.

17. Shared Memory und Copy-Vermeidung

17.1 Warum dies früh relevant wird

Ein zusätzlicher Proxy kann bei großen Tensoren unnötige Kopien erzeugen. Deshalb muss das Produkt Data-Plane-Overhead separat messen und reduzieren.

17.2 Triton Shared Memory

Triton unterstützt System Shared Memory und CUDA Shared Memory; NVIDIA dokumentiert, dass dies in bestimmten Fällen signifikante Performanceverbesserungen ermöglichen kann [S4]. Auf Jetson ist aktuell nur System Shared Memory unterstützt [S4].

17.3 MVP-Stufen

M0

Normales gRPC forwarding. Ziel: Schedulinglogik validieren.

M1

System-Shared-Memory-Referenzen transparent unterstützen.

M2 x86/dGPU

CUDA Shared Memory evaluieren.

M3 Jetson

Optional direktere C-API-/In-Process-Integration evaluieren, da NVIDIA für Edge-Einsatz direkte C-API-Integration empfiehlt [S18].

18. Observability und Kennzahlen

Pflichtmetriken:

```
oneter_requests_received_total
oneter_requests_forwarded_total
oneter_requests_superseded_total
oneter_requests_stale_total
oneter_requests_rejected_infeasible_total
oneter_requests_completed_valid_total
oneter_requests_completed_obsolete_total
oneter_backend_failures_total
```

```

onetimer_deadline_misses_total
onetimer_protected_deadline_misses_total
onetimer_variant_selected_total{variant=...}
onetimer_scheduler_decision_seconds
onetimer_queue_wait_seconds
onetimer_backend_runtime_seconds
onetimer_information_age_seconds
onetimer_stale_compute_seconds_total
onetimer_overload_state

```

18.1 Useful Inference Ratio

```

completed_valid
-----
all completed

```

Je nach Pipeline kann diese Metrik weiter nach Deadline/Freshness spezifiziert werden.

18.2 Stale Compute Waste

Summe Backend/GPU-Zeit fuer Resultate,
die bei Fertigstellung bereits als obsolete gelten

Diese Metrik ist zentral, weil sie den konkreten Ressourcenwert der Garbage-Collector-Idee zeigt.

18.3 Age of Information

p50/p95/p99 pro Modell/Stream. Für kamerabasierte Perception sollte dies neben klassischer Response-Latency eine Hauptkennzahl werden.

19. Benchmark- und Falsifikationsplan

19.1 Baselines

OneTimer darf nicht gegen einen absichtlich schlecht konfigurierten Triton gewinnen.

Mindestens:

1. Direkter Triton, sinnvoll konfiguriert.
2. Triton mit Rate Limiter/Prioritäten, wo relevant [S3].
3. Triton mit passend gewählten Instance Groups.
4. Wenn sinnvoll, Triton Model Analyzer zur Baseline-Optimierung.
5. OneTimer vor derselben Triton-Ausführungseingabe.

19.2 Benchmark-Framework

DISB soll evaluiert und nach Möglichkeit als Basis/Adapter genutzt werden [S16]. Eigene Workload-Generatoren ergänzen Physical-AI-spezifische Freshness-Semantik.

19.3 Erste Modellgruppe

Pragmatisches öffentliches Set:

- Object Detection: eine Modellfamilie mit mindestens zwei Größen;
- Depth;
- Segmentation;

- Pose oder zweiter Vision-Workload;
- später kleines VLM/generative Last.

Wichtig: Der erste Benchmark muss keine perfekte Robotik-Anwendung darstellen. Er muss die Ressourcen- und Schedulingcharakteristik reproduzierbar erzeugen.

19.4 Lastprofile

50 % nominal
75 %
90 %
100 %
110 %
125 %
150 % offered load

Zusätzlich:

- Bursts;
- periodische Protected-Tasks;
- Best-Effort-Bursts;
- stale camera stream;
- variable Backend-Runtime;
- Backend reconnect/failure.

19.5 Kernvergleich A - Freshness

30-FPS-Stream, absichtlich geringere Servicekapazität.

Messen:

- Queue depth;
- Age of Information;
- completed obsolete;
- stale compute;
- effektive aktuelle Inferenzen/s.

19.6 Kernvergleich B - Protected versus Best Effort

Detector/Depth/Pose als Protected/High, VLM-artiger langer Workload als Best Effort.

Messen:

- Protected deadline miss rate;
- Best-Effort p95;
- Gesamtthroughput;
- GPU utilization;
- Variant degradation.

19.7 Kernvergleich C - Varianten

Large + Small derselben logischen Funktion. Last stufenweise erhöhen. Messen:

- Qualitätsmix;
- Deadline Success;
- Anzahl Variant Switches;
- Hystereseverhalten.

19.8 Kill-Kriterien

Projekt pausieren oder Positionierung ändern, wenn nach Baseline-Tuning:

- Deadline-Miss-Reduktion nur marginal ist und stale work kaum ins Gewicht fällt;
- OneTimer unter Normalbetrieb >5 % relevante End-to-End-Regressions verursacht und Shared Memory dies nicht behebt;
- erforderliche Konfiguration pro Kunde so individuell ist, dass ein Engineer wochenlang manuell Profile/Regeln schreiben muss;
- Holoscan/Triton durch einfache vorhandene Konfiguration dieselben Ergebnisse liefern;
- der Nutzen nur in künstlicher Überlast, nicht in realistischen Workloads erscheint.

20. Open-Source- und Lizenzstrategie

20.1 OneTimer Core

Empfehlung:

Apache License 2.0

Warum:

- permissiv;
- verbreitet im Cloud-/AI-/CNCF-Umfeld;
- explizite Patentlizenz im Gegensatz zu sehr minimalistischen Lizenzen;
- OEM-/Enterprise-Adoption wird nicht durch Copyleft blockiert;
- kompatibel mit mehreren geplanten Upstream-Bausteinen.

20.2 Enterprise/Open-Core

Open Source:

- OIP Gateway;
- Scheduling Core;
- Queue Policies;
- Stale Work Collector;
- Basic Profiler;
- Triton Backend Adapter;
- Metrics;
- Benchmarkadapter.

Proprietäre Enterprise-Funktionen können später sein:

- Fleet Profile Registry;
- signierte Policy-/Profile-Bundles;
- RBAC/Audit;
- zentrale Managementkonsole;
- OEM Deployment Tooling;
- Long-Term-Support Builds;
- zertifizierungsnahe Dokumentationspakete;
- Support/SLA.

Wichtig: Der Open-Source-Core muss allein technisch nützlich bleiben. Ein künstlich kastrierter Core würde die gewünschte Entwickleradoption behindern.

20.3 Triton

Der Triton Server ist BSD-3-Clause lizenziert [S1]. OneTimer kann Triton als separates Backend verwenden und, soweit Lizenzbedingungen eingehalten werden, quelloffene Triton-Komponenten referenzieren oder

modifizieren.

MVP-Empfehlung: nicht forken, sondern separat betreiben.

Vorteile:

- deutlich kleinerer Wartungsaufwand;
- Upstream-Security-/Backend-Updates bleiben getrennt;
- klare Architektur;
- leichter Vergleich gegen Baseline.

20.4 Open Inference Protocol

Das OIP-Repository ist Apache-2.0 [S2]. Protobuf/OpenAPI-Spezifikationen können im Rahmen der Lizenz genutzt werden. Lizenz-Header und Notices müssen erhalten bleiben.

20.5 DISB

DISB ist Apache-2.0 [S16]. Benchmarkcode oder Adapter können nach Lizenzprüfung wiederverwendet bzw. geforkt werden. Änderungen müssen klar dokumentiert werden.

20.6 REEF

REEF/Artifact-Code ist Apache-2.0 [S15]. Eine spätere experimentelle Preemption-Komponente kann Techniken oder Code evaluieren. **Nicht MVP**, da die Technologie wesentlich invasiver ist und ihre aktuelle Hardware-/Softwarekompatibilität separat geprüft werden muss.

20.7 NVIDIA TensorRT / NGC

NVIDIA-Binaries und Container unterliegen eigenen NVIDIA-Lizenzbedingungen. OneTimer sollte diese **nicht automatisch als Teil des eigenen Apache-2.0-Artefakts redistributieren**.

Empfohlene Distribution:

OneTimer repository/package

+

Docker Compose / setup instructions referencing official NVIDIA source

Der Anwender lädt das offizielle Triton/TensorRT-Artefakt selbst.

20.8 Lizenz-Allowlist

Initial automatisch erlauben:

Apache-2.0

MIT

BSD-2-Clause

BSD-3-Clause

ISC

Manuelle Prüfung:

MPL-2.0

LGPL variants

EPL variants

Initial blockieren, bis ausdrücklich genehmigt:

GPL
AGPL
SSPL
BUSL
RSAL
Commons Clause
unklare custom/non-commercial licenses

Dies ist eine konservative Engineering-Policy, keine juristische Aussage über generelle Unvereinbarkeit.

21. Lizenz-, Security- und Supply-Chain-Tests

21.1 Rust Dependencies

CI:

`cargo deny check licenses bans advisories sources`

Zusätzlich Third-Party-Notice-Generierung, z.B. via `cargo-about`, sofern die konkrete Toollizenz freigegeben wurde.

21.2 SPDX/REUSE

Repository soll SPDX-Identifizier und maschinenlesbare Copyright-/Lizenzinformationen verwenden.

CI-Ziel:

`reuse lint`

21.3 SBOM

Bei jedem Release:

- SPDX oder CycloneDX SBOM;
- Tool z.B. Syft nach Tool-Lizenzprüfung;
- Artefakt gemeinsam mit Release speichern.

21.4 Vulnerability Scan

- RustSec/cargo-deny;
- Container/Artifact Scan, z.B. Gype oder äquivalent;
- Dependabot/Renovate-artige Updates;
- reproduzierbare Buildinformationen soweit praktikabel.

21.5 FOSS Review vor öffentlichem Release

Vor v1.0:

- alle vendored Sources identifizieren;
- LICENSE/NOTICE erhalten;
- Copyleft-Transitive Dependencies prüfen;
- NVIDIA Redistribution separat prüfen;
- Markenprüfung für "OneTimer" durchführen;
- THIRD_PARTY_NOTICES finalisieren.

22. Security Threat Model MVP

22.1 Angriffsflächen

- gRPC/HTTP Requests;
- Tensor-Metadaten;
- Shared-Memory-Registrierung;
- Konfigurationsdateien;
- Backend/Triton-Verbindung;
- Profil-Dateien;
- Metrics Endpoint.

22.2 Schutzregeln

- maximale Requestgröße;
- maximale Tensoranzahl;
- maximale Dimensionen;
- validierte Zeit-/Deadlinebereiche;
- keine negativen/overflowenden Zeitberechnungen;
- monotonic arithmetic mit checked operations;
- bounded queue size;
- bounded concurrent backend requests;
- timeouts;
- keine Panics aus extern steuerbaren Eingaben.

22.3 Fuzzing

Fuzz-Targets:

- OIP Parameter Normalizer;
- Config Parser;
- Request State Machine;
- Queue Supersession;
- Deadline Arithmetic;
- Profile Parser.

23. Konfigurationsvalidierung und onetimer doctor

Vor Start prüft onetimer doctor:

1. Schema gültig.
2. Backend erreichbar.
3. Modelle existieren in Triton.
4. Varianten besitzen kompatible Shapes/Outputs oder explizite Adapter.
5. Profile vorhanden und nicht stale.
6. Protected-Utilization nicht offensichtlich unmöglich.
7. Queuekapazitäten gesetzt.
8. Stateful Modelle nicht versehentlich LATEST.
9. NEVER_DROP besitzt explizites Overflow-Verhalten.
10. Shared-Memory-Modus auf Plattform unterstützt.

Beispielausgabe:

```
OK Triton reachable at 127.0.0.1:8001
OK detector_large ready
OK detector_small ready
```

```
OK profile matches environment
WARN protected serialized utilization = 82 %
WARN VLM best-effort runtime p99=210 ms; may block protected work
OK all queues bounded
RESULT READY_WITH_WARNINGS
```

24. Arbeitspakete bis Vollausbau

WP0 - Repository, Build, CI, Lizenzgrundlage

Ziel: Ein frischer Clone muss deterministisch gebaut und getestet werden können.

Aufgaben:

- Rust Workspace anlegen.
- Format/Lint: `cargo fmt`, `cargo clippy`.
- Unit-Test-Framework.
- GitHub Actions oder äquivalente CI.
- Apache-2.0 LICENSE.
- NOTICE / THIRD_PARTY_NOTICES Platzhalter.
- `cargo-deny` und `deny.toml`.
- REUSE-Konfiguration.
- Security Policy.

Definition of Done:

```
git clone
cargo build --workspace
cargo test --workspace
cargo deny check ...
reuse lint
```

laufen in sauberer CI erfolgreich.

WP1 - Discrete Event Scheduler Simulator

Ziel: Scheduling-Theorie vollständig ohne GPU testbar machen.

Aufgaben:

- Monotonic Simulated Clock.
- RequestDescriptor.
- ModelContract.
- RuntimeProfile.
- Events: Arrival, DispatchComplete, Timer, BackendFailure.
- deterministischer Replay.

DoD: 100.000+ synthetische Ereignisse deterministisch reproduzierbar; gleicher Seed ergibt identische Reihenfolge und Kennzahlen.

WP2 - Queue Policies und Stale Work Collector

Aufgaben: Latest, LatestPerKey, FIFO, NeverDrop.

Pflichttests:

- LATEST queued depth ≤ 1 .
- neuer Request supersediert alten queued Request.
- running/forwarded wird nicht fälschlich als abgebrochen angenommen.

- FIFO erhält Reihenfolge.
- NeverDrop wird niemals durch Supersession entfernt.
- Overflow löst definierte Reaktion aus.

WP3 - Deadline, Slack, Feasibility

Aufgaben:

- absolute Deadline aus Generation Time;
- Slack pro Variante;
- infeasible detection;
- EDF/Criticality ordering;
- Look-ahead horizon;
- periodic arrival forecast.

DoD: Golden Tests und Property Tests zeigen, dass der Scheduler keinen niedrigeren Request wesentlich startet, wenn die konservative Simulation dadurch einen Protected-Miss prognostiziert und eine sichere Alternative existiert.

WP4 - Schedulability/Doctor Analyzer

Aufgaben:

- einfache Utilization-/Demandwarnungen;
- unplausible Contracts erkennen;
- Blockingwarnungen;
- klare Diagnostics.

WP5 - Overload State Machine

Aufgaben: NORMAL, FRESHNESS_PRESSURE, DEGRADED, REJECT_BEST_EFFORT, PROTECTED_ONLY.

Tests: Hysterese, Recovery, keine schnellen Oszillationen.

WP6 - Model Variant Resolver

Aufgaben:

- logical -> physical mapping;
- quality sorting;
- min quality;
- best feasible;
- variant hysteresis;
- stateful pinning.

WP7 - OIP gRPC Gateway - transparente Stufe

Ziel: Standardclient kann über OneTimer inferieren, zunächst FIFO/Forwarding.

Aufgaben: ServerLive/Ready, ModelReady, ModelMetadata, ModelInfer.

DoD: Ein existierender kompatibler Client kann ohne kundenspezifisches SDK einen Triton-Testserver über OneTimer ansprechen.

WP8 - Triton Backend Adapter

Aufgaben:

- Connection Pool;
- async inference;
- error mapping;
- cancellation soweit möglich;
- health/reconnect;
- backend latency timing.

Wichtig: Backend kennt keine fachliche Schedulingpolitik.

WP9 - Echtzeit-Policies im Gateway

Scheduler aus WP1-6 in echten Gatewaypfad integrieren.

DoD: LATEST-Requestfolge gegen echten Mock-/Triton-Backend erzeugt erwartete Supersession ohne Race Condition.

WP10 - Single-Model Profiler

CLI:

```
onetimer profile -c config.yaml
```

Aufgaben: Warmup, definierte Samples, p50/p95/p99, Model/Environment Fingerprint.

WP11 - Online Runtime Estimator

Rolling statistics, safety margin, degraded profile.

Tests: Sprunghafter Runtimeanstieg führt kontrolliert zu konservativerer Planung.

WP12 - Interference Profiler

Automatisiert ausgewählte Paare messen.

DoD: Matrix serialisiert und vom Scheduler ohne Online-Profiling verwendet.

WP13 - Metrics / Prometheus

Alle Kernmetriken plus strukturierte State-Transition-Events.

WP14 - Shared-Memory Compatibility

Zuerst System Shared Memory. Onetimer darf Regionen/Offsets nicht kopieren, wenn transparentes Forwarding möglich ist.

Plattformtest: Jetson erwartet nur system shared memory [S4].

WP15 - Benchmark Harness / DISB Adapter

DISB evaluieren und adaptiertes Modul einbinden, sofern Lizenz-/Kompatibilitätstest bestanden [S16].

WP16 - Tuned Triton Baseline

Scripts/Config für:

- Rate Limiter;
- Priorities;
- Instance Groups;

- Perf Analyzer/Model Analyzer, soweit passend.

DoD: Dokumentierter Baseline-Tuning-Report. Kein Strawman.

WP17 - Freshness Benchmark

30-FPS-artiger Stream mit absichtlich <30 FPS Servicekapazität. Ergebnisse als JSON/CSV/Plot.

WP18 - Protected-vs-Best-Effort Benchmark

Mehrere periodische Modelle plus langlaufende Hintergrundlast.

WP19 - Variant Benchmark

Qualitäts-/Deadline-Frontier und Hysterese.

WP20 - Stress, Property, Fuzz

- 10 Mio. synthetische Scheduler Events;
- gRPC disconnects;
- malformed params;
- backend timeouts;
- queue pressure;
- fuzzing.

WP21 - Docker/Quickstart/Product Preview

- OneTimer Image;
- Compose, das offizielle Triton-Image referenziert;
- Example models getrennt nach Lizenz;
- 5-Minuten-Quickstart.

WP22 - Jetson Port

Erst nach positivem x86/dGPU-Go-Gate.

- Orin-Profil;
- Shared Memory;
- CPU/RAM Overhead;
- p99 Scheduler;
- realistische Power Modes nur messen, nicht thermisch steuern.

WP23 - HTTP/OIP

REST zusätzlich zu gRPC.

WP24 - Stateful/Sequence Support

Triton Sequence-Batching-Semantik korrekt durchreichen; stateful Queues und Varianten pinnen.

WP25 - DAG / End-to-End Deadlines

Pipeline:

camera -> detector -> tracker -> VLM

End-to-End Budget auf Teilstufen verteilen. Aktuelle Robotikforschung wie RED untersucht adaptive Echtzeit-DAG-Scheduling-Ansätze [S19].

WP26 - Cooperative VLM/LLM Adapter

Token-/Chunk-Yield-Points, backendabhängig. Keine universelle harte Preemption behaupten.

WP27 - Optional Native Executor

Nur falls Benchmark zeigt, dass Triton-Gatewaykontrolle den Nutzen begrenzt.

- C++/TensorRT oder Triton C API;
- direktere Datenpfade;
- kontrollierte Streams.

WP28 - Experimental Preemption Backend

REEF-Techniken evaluieren [S15]. Separater experimenteller Build; nicht Default.

WP29 - Multi-Accelerator

GPU/DLA/CPU/NPU als zusätzliche Ausführungsdimension.

WP30 - Fleet/Enterprise

- zentrale Profile;
- signierte Konfiguration;
- RBAC/Audit;
- Rollout Policies;
- LTS;
- On-Prem Management.

25. Milestones und harte Go-Gates

Milestone M0 - Theorie

WP0-WP6. Keine GPU notwendig.

Gate: Simulator zeigt korrekte Freshness-, Deadline- und Overload-Invarianten.

Milestone M1 - Transparent Proxy

WP7-WP9.

Gate: OIP-Kompatibilität funktioniert und End-to-End-Inferenz über OneTimer ist funktional.

Milestone M2 - Profiling

WP10-WP14.

Gate: OneTimer kann reale Runtime-/Interference-Profile reproduzierbar erzeugen und verwenden.

Milestone M3 - Falsifikation

WP15-WP20.

Gate: Mindestens eines der definierten Hauptziele wird gegen tuned Triton erreicht. Sonst Projektpositionierung pausieren.

Milestone M4 - Product Preview

WP21-WP22.

Gate: 5-Minuten-Quickstart auf mindestens x86/dGPU und Jetson-Demoumgebung.

Milestone M5 - Erweiterungen

WP23+ nur nach positivem Produktkern.

26. Coding-Agent-Protokoll

26.1 Grundregel

Ein Coding-Agent darf nicht versuchen, das gesamte System in einem großen Commit zu erzeugen. Jede Stufe muss kompiliert, getestet und durch eine kleine technische Entscheidung abgesichert werden.

26.2 PR-/Commit-Reihenfolge

1. Workspace + CI + Lizenzprüfung.
2. Core Types + Sim Clock.
3. FIFO Queue.
4. Latest Queue.
5. LatestPerKey.
6. NeverDrop + bounded overflow.
7. Request State Machine.
8. Deadline/Slack.
9. EDF/Criticality.
10. Look-ahead simulator.
11. Variant Resolver.
12. Overload State Machine.
13. Property Tests.
14. OIP gRPC transparent proxy.
15. Triton backend adapter.
16. OneTimer policies in live path.
17. Profiler.
18. Online estimator.
19. Interference matrix.
20. Metrics.
21. Benchmark adapter.
22. Tuned Triton baseline.
23. Go/No-Go benchmark.

Keine CUDA-/Native-Executor-Arbeit vor erfolgreichem Go-Gate.

26.3 Definition of Done fuer jeden Coding-Agent-Task

Jeder Task benötigt:

- Code;
- Unit Tests;
- mindestens einen negativen Test;
- keine neue Warnung in cargo clippy;
- Lizenzcheck der neuen Dependency;
- Dokumentationsupdate;
- explizite Annahmen;
- Benchmark nur dort, wo Performance relevant ist.

26.4 Coding-Regeln

- Kein unwrap()/expect() auf extern kontrolliertem Hot-Path ohne technisch begründete Unmöglichkeitsinvariante.
- Keine unbounded Tokio channels.
- Keine unbounded Vec/HashMap-Aufnahme aus Remotegrößen.
- Checked integer/time arithmetic.
- Keine Wall Clock für Deadlines.
- Keine globale Mutex-geschützte Mega-State-Struktur.
- Keine DB im Hot Path.
- Kein LLM/RL im Hot Path.
- Kein Logging des vollständigen Tensorpayloads.
- Keine stille Fehlerkorrektur bei ungültigen Contracts.

27. Master Prompt fuer einen LLM-Coder

Der folgende Text kann als Ausgangsanweisung für einen Coding-Agenten verwendet werden. Er ersetzt nicht die einzelnen Arbeitspakete, sondern setzt deren Ausführungsregeln.

You are implementing Vigilant OneTimer, an Apache-2.0 open-core inference governor.

Product architecture:

- OneTimer is NOT a new inference engine in the MVP.
- It is a Rust gateway/governor in front of NVIDIA Triton.
- It speaks Open Inference Protocol v2 and forwards admitted requests to Triton.
- Triton remains the inference execution backend.

Primary product goals:

1. Eliminate queued inference work that has become obsolete.
2. Protect latency-sensitive periodic inference from lower-value work.
3. Reject or delay work that is no longer feasible before dispatch.
4. Select the highest-quality feasible model variant.
5. Remain compatible with existing inference clients and models.

Never assume hard GPU preemption.

Never use an LLM or reinforcement learning in the scheduling hot path.

Never create unbounded queues.

Never silently drop NEVER_DROP work.

Never add CUDA/native TensorRT execution before the proxy benchmark gate proves it is necessary.

Implementation order:

- Work only on the currently assigned WP/PR.
- Read the architecture and invariants before modifying code.
- Write deterministic unit/property tests first for scheduler semantics.
- Keep onetimer-core free of networking and Triton dependencies.

- Keep backend policy out of the Triton adapter.
- Every external input must be validated and bounded.
- Every dependency must pass license checks.

Required checks before completing a task:

```
cargo fmt --check
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo test --workspace
cargo deny check licenses bans advisories sources
reuse lint
```

When a requirement is ambiguous, choose the smallest implementation that preserves the documented product invariant. Do not expand scope without a written ADR and benchmark reason.

28. Testkatalog

28.1 Golden Test G-001 - Latest supersedes queued

```
r1 arrives -> queued
r2 arrives same key before r1 dispatched
expected: r1 SUPERSEDED, r2 QUEUED
```

28.2 G-002 - Running is not magically cancelled

```
r1 FORWARDED
r2 arrives same key
expected: r1 remains FORWARDED, r2 QUEUED
completion of r1 may become COMPLETED_OBSOLETE
```

28.3 G-003 - FIFO order

r1,r2,r3 -> completion order respects queue order unless explicit backend error

28.4 G-004 - NeverDrop overflow

Bei voller Queue kein stilles Löschen. Erwartung: Backpressure/explicit reject according configured overflow policy.

28.5 G-005 - Deadline from generation time

Ein bereits 25 ms alter Frame mit 30-ms-Deadline erhält nicht noch einmal 30 ms ab Gateway Arrival.

28.6 G-006 - Variant

```
remaining budget 14 ms
large p99 20
medium p99 12
small p99 7
expected medium
```

28.7 G-007 - Known future protected arrival

Best-Effort darf nicht gestartet werden, wenn konservative Blockingannahme den erwarteten Protected-Request sicher gefährdet und sinnvolles Warten möglich ist.

28.8 G-008 - Overload bounded

150 % offered load für definierte Dauer. Keine Queue wächst über Capacity. Prozess bleibt responsive.

28.9 G-009 - Runtime degradation

Profil 10 ms, beobachtet über Fenster 20 ms. Safety Margin/online estimator reagiert; Scheduler verwendet ggf. kleinere Variante.

28.10 G-010 - Profile stale

Model hash/Backendversion geändert. Altes Profil darf nicht stillschweigend als exakt gültig gelten.

28.11 G-011 - Stateful cannot Latest

Config mit stateful: true und unzulässigem Supersession-Modus wird durch doctor blockiert oder explizit als unsupported markiert.

28.12 G-012 - Time overflow

Extremwerte in Clientparametern dürfen keinen Integeroverflow/negative Deadline erzeugen.

29. Performanceoptimierungsplan

Prioritäten strikt in dieser Reihenfolge:

29.1 Nicht rechnen

Die beste Optimierung ist eine Inferenz, die keinen Wert mehr hat und deshalb nie gestartet wird.

29.2 Nicht zu frueh rechnen

Best-Effort-Arbeit nicht starten, wenn sie in wenigen Millisekunden einen Protected-Job blockieren könnte.

29.3 Richtige Variante

Nicht Large starten, wenn Medium die Qualitätsanforderung erfüllt und Large die Deadline gefährdet.

29.4 Konkurrenz kennen

Interference Matrix statt blindem parallelem Start.

29.5 Kopien vermeiden

Shared Memory/Data Plane optimieren.

29.6 Hot Path vereinfachen

Lookup + bounded look-ahead; keine teuren Solver online.

29.7 Erst danach Mikrooptimieren

Allocation, Cache Layout, branch prediction usw. erst messen, nicht spekulativ optimieren.

30. Stabilitätsoptimierungsplan

30.1 Deterministischer Core

Netzwerk- und Backend-I/O von Scheduling-Core trennen.

30.2 Replayability

Optionalen Scheduler Event Trace schreiben können:

```
arrival  
supersession  
state transition  
dispatch decision  
completion
```

Ein Trace muss offline im Simulator reproduzierbar sein. Das ist für komplexe Race-/Überlastfehler extrem wertvoll.

30.3 Profile Circuit Breaker

Wenn reale Laufzeiten über mehrere Samples deutlich außerhalb der Profilannahmen liegen:

1. Profilstatus DEGRADED;
2. Margin erhöhen;
3. große Variante aus automatischer Auswahl nehmen;
4. Metrik/Alarm;
5. optional Reprofile-Empfehlung.

30.4 Backend Circuit Breaker

Bei Tritonfehlern:

- begrenzte Retries nur, wenn fachlich noch sinnvoll;
- kein Retry eines inzwischen stale Requests;
- Exponential Backoff außerhalb Protected Hot Path;
- Health Status.

30.5 Config Atomicity

Live Config Reload später nur atomar:

```
parse -> validate -> build new immutable config snapshot -> swap
```

Nie halbfertige neue Contracts im laufenden Scheduler.

31. Was Open Source uebernommen werden sollte - und was nicht

31.1 Direkt sinnvoll

Open Inference Protocol

Spezifikation/Protobuf als Standardinterface nutzen [S2].

Triton als Execution Backend

Nicht kopieren, sondern als separaten Server verwenden [S1].

DISB

Benchmarklogik/Workloadadapter prüfen und wiederverwenden [S16].

31.2 Nur als Forschung/Referenz

REEF

Algorithmen und später mögliche Preemption evaluieren, aber nicht zum MVP-Fundament machen [S15].

EdgeServing

Stability-/future-queue-Ideen als Forschungsreferenz für späteren Scheduler nutzen [S9]. Kein 1:1-Nachbau ohne eigenen Benchmarkgrund.

Holoscan

Als Konkurrenz-/Integrationsreferenz nutzen. Latest-frame und parallele Multi-Model-Muster zeigen, was bereits Standard ist [S7][S8].

31.3 Nicht sinnvoll im MVP

- eigener CUDA-Kernel-Scheduler;
- TensorRT neu wrappen, bevor die Gatewaygrenze bewiesen ist;
- eigenes Modellformat;
- eigenes Robotik-Middleware-System;
- eigener ROS2-Ersatz.

32. Produktverpackung und Developer Experience

32.1 README Above the Fold

Vigilant OneTimer
Adaptive inference QoS for shared edge GPUs.

Keep Triton. Keep your models.
OneTimer drops stale work, protects latency-sensitive inference,
and chooses the best feasible model variant under load.

Darunter sofort:

```
docker compose up -d
onetimer doctor -c onetimer.yaml
onetimer profile -c onetimer.yaml
onetimer serve -c onetimer.yaml
```

32.2 Demo

Eine gute öffentliche Demo zeigt nicht abstrakte Schedulergraphen, sondern zwei synchronisierte Videos/Timelines:

Baseline: Detector arbeitet bei VLM-Burst zunehmend alte Frames ab.

OneTimer: Detector überspringt veraltete Arbeit, bleibt näher am aktuellen Bild; VLM wird verzögert/degradiert.

Darunter live:

AoI p95
Protected deadline success
stale compute
VLM latency

33. Go-to-Market-Story

33.1 Was nicht verkauft wird

Nicht:

“Wir haben einen besseren GPU-Scheduler.”

Das ist zu intern und leicht als Eigenentwicklung abzutun.

33.2 Was verkauft wird

Run more AI on the same edge GPU without letting stale or low-value work delay what matters.

Für Robotik:

Your VLM should never make perception old.

Für Engineering:

Keep Triton and your existing models. Add freshness and deadline contracts instead of hand-tuning cross-model GPU contention.

Für OEM/Management:

Stabilere KI-Latenz unter Last, weniger verschwendete Inferenz und potenziell höhere Funktionsdichte auf derselben Compute-Plattform.

34. Design-Partner-Fragen

Nicht fragen:

“Braucht ihr einen Scheduler?”

Besser:

1. Wie viele neuronale Modelle laufen gleichzeitig auf derselben Edge-GPU?
2. Gibt es Streams, bei denen nur der neueste Input zählt?

3. Was passiert aktuell bei GPU-Überlast?
4. Werden alte Frames abgearbeitet oder gedroppt?
5. Gibt es periodische Perception-Deadlines?
6. Wie verhindert ihr, dass VLM/Reasoning andere Perception stört?
7. Habt ihr mehrere Modellgrößen/Quantisierungen derselben Funktion?
8. Wie viel Engineering steckt in CUDA Streams, Triton Config, Rate Limits und Frequenzsteuerung?
9. Habt ihr bereits eine interne cross-model admission control?
10. Würdet ihr einen Drop-in-OIP-Proxy testen, wenn er gegen euren aktuellen Stack reproduzierbar >2x weniger Deadline Misses zeigt?

35. Risiken

R1 - Problem existiert, aber interne Eigenentwicklung ist "gut genug"

Mitigation: Installation und Benchmark müssen extrem einfach sein; Vorteil muss groß sein.

R2 - Triton/Holoscan schliessen Featureluecke

Mitigation: standardbasiert bleiben; Innovation auf Policy/Profile/automation statt einzelne Features konzentrieren.

R3 - Proxy-Overhead vernichtet Vorteil

Mitigation: Shared Memory; in-process/native executor nur bei Messbeleg.

R4 - Varianten sind fachlich nicht austauschbar

Mitigation: Varianten optional, Output Contract Validation, user-supplied quality.

R5 - GPU-Preemption zu begrenzt

Mitigation: OneTimer-Design basiert gerade auf Pre-dispatch-Admission, nicht auf unrealistischer Preemption.

R6 - Realer Workload besitzt kaum stale Arbeit

Mitigation: Kill-Kriterium. Dann ist Freshness nicht der Wedge und Produkt muss neu bewertet werden.

R7 - Safety/Automotive Erwartungen

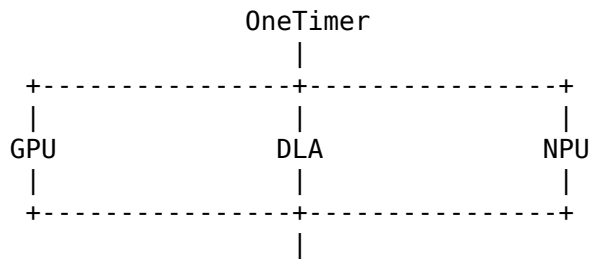
Mitigation: keine Safety-Zertifizierung im MVP behaupten; Robotik/Edge-Engineering zuerst.

R8 - Lizenz-/Redistributionsfehler

Mitigation: NVIDIA-Binaries nicht bundlen; automatisierte Lizenzchecks; Counsel vor Release.

36. Vollausbauvision

Nach erfolgreich bewiesenem Core kann OneTimer zu einer allgemeineren **Inference QoS Control Plane for Physical AI** wachsen:



Models: detector, depth, pose, segmentation, VLM, audio, planning

Constraints:

- deadline
- freshness
- criticality
- quality
- memory
- later energy/thermal

Actions:

- execute
- wait
- supersede
- reject
- degrade
- switch variant
- switch accelerator
- chunk/yield

Thermik bleibt auch hier eine **spätere Constraint-Quelle**, nicht der ursprüngliche Produktkern.

37. Endgültige Produktentscheidung

Für den ersten Entwicklungszyklus wird festgelegt:

Product:	Vigilant OneTimer
Category:	Adaptive inference governor / inference QoS
Manufacturer:	Vigilant e.K., Stuttgart, Germany
Core language:	Rust
Backend MVP:	NVIDIA Triton as separate execution server
Protocol MVP:	Open Inference Protocol v2 gRPC
Hardware MVP:	single NVIDIA GPU; Jetson after x86/dGPU proof
Core policies:	LATEST, LATEST_PER_KEY, FIFO, NEVER_DROP
Criticality:	PROTECTED, HIGH, NORMAL, BEST_EFFORT
Core mechanisms:	stale work collection deadline/slack admission control periodic look-ahead model variant selection runtime profiles interference profiles overload degradation
Thermal control:	OUT
Energy scheduling:	OUT
TANS:	OUT
Hard realtime claim:	OUT
Native CUDA engine:	OUT until benchmark proves need

License core: Apache-2.0 recommended
Business model: Open Core + Enterprise/OEM/Support

38. Die eine entscheidende Hypothese

Wenn mehrere AI-Workloads eine gemeinsame Edge-GPU teilen, kann ein protokollkompatibler Governor durch Freshness-bewusste Arbeitselimination, konservative Zulassung und Variantenwahl erheblich mehr rechtzeitige und nützliche Inferenz liefern als ein gut konfigurierter generischer Serving-Stack - ohne die Kundenmodelle oder den Ausführungsbackend zu ersetzen.

Diese Hypothese wird nicht durch weitere Dokumentation bewiesen. Sie wird durch M3-Benchmarks bewiesen oder falsifiziert.

Anhang A - Quellen und Recherchebasis

[S1] NVIDIA Triton Inference Server - GitHub / BSD-3-Clause

<https://github.com/triton-inference-server/server>

[S2] KServe Open Inference Protocol - Specification / Apache-2.0

<https://github.com/kserve/open-inference-protocol>

[S3] NVIDIA Triton - Rate Limiter

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/rate_limiter.html

[S4] NVIDIA Triton - Shared-Memory Extension

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/protocol/extension_shared_memory.html

[S5] NVIDIA CUDA Programming Guide - Stream Priorities / no guaranteed preemption of already running work

<https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/asynchronous-execution.html>

[S6] NVIDIA Triton - Request Cancellation

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/request_cancellation.html

[S7] NVIDIA Holoscan - Schedulers / Async Buffer latest-frame semantics

<https://docs.nvidia.com/holoscan/sdk-user-guide/components/schedulers>

[S8] NVIDIA Holoscan - Inference Operator / parallel models without automatic resource sufficiency check

<https://docs.nvidia.com/holoscan/sdk-user-guide/operators/inference>

[S9] EdgeServing: Deadline-Aware Multi-DNN Serving at the Edge, 2026

<https://arxiv.org/abs/2605.05527>

[S10] Figure AI - Staff AI Inference & Acceleration Engineer

<https://job-boards.greenhouse.io/figureai/jobs/4692572006>

[S11] Agile Robots / Idealworks - Edge AI Stack, 24 July 2026

<https://www.agile-robots.com/en/news/detail/a-gentle-introduction-to-agile-robots-and-idealworks-edge-ai-stack-for-maximum-compatibility/>

[S12] NEURA Robotics - Robot Perception Expert / Jetson & CUDA

<https://jobs.neura-robotics.com/offer/robot-perception-expert-human/4aaf7184-bcf1-470e-be3d-280ca25afa0c>

[S13] ARX Robotics - Staff Engineer Robotics Perception

<https://job-boards.eu.greenhouse.io/arxroboticsgmbh/jobs/4875205101>

[S14] International Federation of Robotics - World Robotics 2025 / Germany installations

<https://ifr.org/worldrobotics/report-2025>

[S15] REEF - Real-time GPU-accelerated DNN inference scheduling / Apache-2.0

<https://github.com/SJTU-IPADS/reef>

[S16] DISB - DNN Inference Serving Benchmark / Apache-2.0

<https://github.com/SJTU-IPADS/disb>

[S17] NVIDIA Triton - Stateful sequence client/background documentation

<https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/>

[S18] NVIDIA Triton - Jetson Support

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/jetson.html

[S19] RED - Adaptive real-time DAG scheduling for robotic inference, 2026

<https://arxiv.org/abs/2605.24044>

Anhang B - Begriffsdefinitionen

Admission Control: Entscheidung vor Dispatch, ob eine Aufgabe zugelassen, verschoben, degradiert oder verworfen wird.

Age of Information (AoI): Alter der zugrundeliegenden Information bei Nutzung/Fertigstellung, nicht nur Serverlatenz.

Backend: Ausführungsserver, im MVP NVIDIA Triton.

Best Effort: Arbeit, die verfügbare Reserve nutzen darf, aber Protected-Ziele nicht wissentlich gefährden soll.

Criticality: Produktinterne Wichtigkeitsklasse. Im MVP keine Safety-Zertifizierung.

Deadline: Zeitpunkt, bis zu dem eine Inferenz gemäß Contract abgeschlossen sein sollte.

Feasible: Nach konservativer Planung innerhalb der relevanten Verträge ausführbar.

Freshness: fachliche Aktualität des Inputs.

Governor: Steuerungsschicht, die über Zulassung, Reihenfolge und Varianten entscheidet, aber die eigentliche neuronale Berechnung nicht selbst implementieren muss.

LATEST: Queue-Policy, bei der nur der aktuellste queued Request pro Scope erhalten bleibt.

Logical Model: Kundenfunktion wie detector, unabhängig von der physischen Variante.

Physical Variant: konkrete Backendmodellversion wie detector_small.

Protected: höchste OneTimer-MVP-Schutzklasse für zeitkritische Arbeit; keine hard-real-time Garantie.

Slack: verbleibende Zeit bis Deadline abzüglich konservativer Ausführungszeit.

Stale: fachlich zu alt, um noch sinnvoll verarbeitet/geliefert zu werden.

Superseded: durch einen neueren Request desselben Freshness-Scope ersetzt.

Useful Timely Inference: Inferenz, deren Ergebnis sowohl fachlich noch aktuell als auch innerhalb des relevanten Zeitvertrags verfügbar ist.

Anhang C - Release-Checkliste v0.1

- ☐ Apache-2.0 LICENSE vorhanden.
- ☐ Markencheck "OneTimer" vor öffentlicher Produktvermarktung.
- ☐ THIRD_PARTY_NOTICES vollständig.
- ☐ Keine ungeprüfte Redistribution von NVIDIA-Binaries.
- ☐ OIP-Lizenzheader erhalten.
- ☐ cargo deny grün.
- ☐ reuse lint grün.
- ☐ SBOM erzeugt.
- ☐ Vulnerability Scan grün bzw. dokumentierte Ausnahmen.
- ☐ OIP Compatibility Tests grün.
- ☐ Golden Scheduler Tests grün.
- ☐ 10M Stress Events ohne Leak/Deadlock.
- ☐ Tuned Triton Baseline reproduzierbar.
- ☐ Benchmarkwerte als Ziele oder gemessene Werte korrekt gekennzeichnet.
- ☐ Keine hard-real-time/Safety-Marketingbehauptung.
- ☐ Quickstart auf sauberer Maschine validiert.
- ☐ Known Limitations dokumentiert.

Anhang D - OneTimer v0.1 Erfolgskriterien auf einer Seite

OneTimer v0.1 ist **technisch erfolgreich**, wenn:

1. Ein Standard-OIP-Client durch Endpointwechsel über OneTimer und weiter zu Triton inferieren kann.
2. LATEST und LATEST_PER_KEY alte queued Sensorarbeit zuverlässig supersedieren.
3. Deadlines auf Generation Time basieren.
4. Protected-Requests durch Admission/Look-ahead nachweislich besser geschützt werden.
5. Varianten automatisch nach Quality/Feasibility ausgewählt werden.
6. alle Queues bounded sind und Überlast nicht zum Speicherwachstum führt.
7. Shared-Memory-Pfad oder ein anderer Weg den Proxy-Overhead hinreichend klein hält.
8. gegenüber tuned Triton mindestens ein großer Benchmarkeffekt erreicht wird: 2x weniger Protected Deadline Misses oder 30 % weniger stale compute, idealerweise beides.
9. das System ohne kundenspezifischen Fork installierbar bleibt.
10. Lizenz- und Supply-Chain-Pipeline einen öffentlich distributierbaren Apache-2.0-Core ermöglicht.

OneTimer v0.1 ist **geschäftlich noch nicht validiert**, bis mindestens mehrere externe Engineering-Teams bestätigen, dass das Problem in ihrem Stack existiert und ein Drop-in-Produkt einem weiteren internen Spezialmodul vorzuziehen wäre.